



POLITECHNIKA
LUBELSKA
WYDZIAŁ ELEKTROTECHNIKI
I INFORMATYKI

Praca dyplomowa inżynierska

na kierunku Informatyka

w specjalności Inżynieria oprogramowania

Dwuwymiarowa gra platformowa z elementami fabularnymi
Platform game with storytelling elements in a 2D environment

Jan Drewienkowski
99523

Szymon Ciechoński
97578

Promotor dr. Inż. Jakub Smółka

Lublin rok 2025

Spis Treści

Streszczenie	4
Abstract	4
1. Wstęp	4
2. Cel i zakres pracy	5
2.1. Cel pracy	6
2.2. Zakres pracy	6
2.3. Podział pracy	6
3. Analiza istniejących rozwiązań	7
3.1. Analiza rynku gier platformowych	8
3.2. Analiza rynku gier fabularnych	9
3.3. Analiza rynku gier zaimplementowanych za pomocą Godot Engine	10
3.4 Analiza rynku na przykładzie wybranych gier	11
4. Opis wymagań systemu	15
5. Narzędzia zastosowane w projekcie	16
6. Fabuła gry	19
7. Opis postaci	24
8. Projekt gry	31
9. Elementy grafiki	41
10. Implementacja głównych mechanik	52
11. Elementy dźwiękowe	84
12. Testy i debugowanie	88
13. Wnioski	89
Bibliografia	89

Streszczenie

Gry komputerowe to coraz częściej nieodłączna część codziennego życia dużej ilości osób, zgodnie z tym co piszą Rachel Kowert i Thorstem Quandt w swojej książce [1], dlatego celem tej pracy inżynierskiej było stworzenie platformowej gry dwuwymiarowej z elementami fabularnymi. Projekt łączy cechy gry z widokiem z boku i gry z systemem walki turowej w formacie klasycznych gier fabularnych. Szczególnie ważnym aspektem pracy autorów jest wcześniej wspomniany element historii przedstawionej dla gracza i w jaki sposób jest mu ona pokazana [2]. Autorzy dokonali analizy istniejących na rynku rozwiązań podobnych do tworzonego przez nich projektu oraz przedstawili wymagania funkcjonalne dotyczące tworzonej gry. Następnie nakreślili charakter gry poprzez stworzenie jej fabuły i diagramów ukazujących wymagania funkcjonalne w bardziej szczegółowy sposób. Po zakończeniu etapu projektowania samego projektu, autorzy pokazali rezultaty jego implementacji. Wykorzystując silnik Godot i system wtyczek dołączonych do aplikacji, foldery zawierające pliki obrazów i skryptów zostały ułożone w interaktywny program, pozwalający na doświadczenie historii jak i zabawy.

Abstract

Computer games are increasingly becoming an integral part of everyday life for many people, according to Rachel Kowert and Thorstem Quandt in their book [1]. Therefore, the aim of this engineering thesis was to create a two-dimensional platform game with story elements. The project combines the features of a side-scroller game and a turn-based combat system in the format of classic role-playing games. A particularly important aspect of this work is the aforementioned element of the story presented to the player and how it is shown to them [2]. The authors analyzed existing solutions on the market similar to their project and presented the functional requirements for the game they were creating. They then outlined the nature of the game by creating its storyline and diagrams showing the functional requirements in more detail. After completing the design phase of the project itself, the authors showed the results of its implementation. Using the Godot engine and the plugin system included with the application, folders containing image and script files were arranged into an interactive program that allows the player to experience the story and have fun.

1. Wstęp

Na rynku gier komputerowych można zauważyć duże ilości gier dwuwymiarowych, zarówno mających swoją premierę kilkadziesiąt lat temu, jak i wiele nowszych tytułów. Praca ma na celu podtrzymanie tej tradycji, tworząc grę nawiązującą do różnych pozycji tego gatunku. Poprzez tą grę twórcy chcieliby wprowadzić gracza w świat będący połączeniem gier, które można skategoryzować jako grę z widokiem z boku oraz klasyczna grę fabularną. Przykładem tutaj może być “Mario” [3], “Fear & Hunger” [4] i “Earthbound” [5].

Oznacza to w praktyce zaprojektowanie gry, gdzie postać gracza będzie miała możliwość pokonywania poziomów mapy w prawą lub lewą stronę, a walka z przeciwnikami będzie przebiegać poprzez wybieranie opcji czynności w oknie interfejsu gry.

Jedną z podstawowych części tej pracy jest zapewnienie grze bogatej fabuły, zgrywającej się w integralną całość i trzymającej gracza w zaciekawieniu. Jej całokształt ma komplementować dźwięk i wygląd gry, tak aby doświadczenie gracza poznającego historię było jak najbardziej immersyjne.

Do zrealizowania tego celu twórcy posługują się silnikiem Godot [6] będącym sprawdzonym rozwiązaniem otwarto-źródłowym. Obsługuje on między innymi swój dedykowany język programowania GDscript [7], ale umożliwia on również korzystanie z C++ [8] i C# [9]. W trakcie implementowania kodu autorzy brali inspirację z książki “Programming For Game Design” [10]. Zdecydowaliśmy się na ten silnik ze względu na prężnie rozwijającą się społeczność, jak i łatwość użycia.

Główną motywacją do wybrania takiego tematu pracy inżynierskiej jest fascynacja wcześniej wspomnianymi grami dwuwymiarowymi jej autorów. Chcieliby oni przedstawić graczowi ich własną wizję gier dwuwymiarowych, w taki sposób, aby chciał powrócić do innych gier tego gatunku.

2. Cel i zakres pracy

2.1. Cel pracy

Celem pracy inżynierskiej jest skonstruowanie dwuwymiarowej gry ze zbalansowanym turowym systemem walki [11] posiadającej elementy platformowe w silniku Godot. Istotnym aspektem jest element fabularny, który podczas rozgrywki przedstawi graczowi historię o problemach życia pracownika korporacji. Cały program będzie się składał z map platformowych, stworzonych w stylu gier “Super Mario Bros” [3], na których jest możliwość walki z przeciwnikami oraz z map fabularnych, gdzie gracz może rozpocząć dialog z innymi postaciami. Wszystkie poczynania zdecydują, o tym które zakończenie gracz dostanie pod koniec rozgrywki.

2.2. Zakres pracy

Zakres pracy obejmuje:

- Analiza rynku i istniejących rozwiązań
- Analiza i dobór narzędzi programistycznych
- Zdefiniowanie wymagań funkcjonalnych i нефункциональных
- Przygotowanie diagramów opisujących wymagania
- Opracowanie fabuły i napisanie dialogów
- Implementacja gry
- Debugowanie gry
- Testowanie gry
- Wprowadzenie poprawek
- Opracowanie wniosku

2.3. Podział pracy

2.3.1. Jan Drewienkowski:

- Menadżer dialogów
- Menu ekwipunku
- System walki turowej
- Rysowanie części zestawów kafelków i postaci
- Dialog i fabuła
- Tworzenie map

- Wygląd postaci gracza
- Tworzenie HUD i interfejsu gracza

2.3.2. *Szymon Ciechoński:*

- Menadżer scen i drzwi
- Maszyna stanów
- Menu główne
- Zapis gry
- Łączenie gałęzi projektu w całość
- Rozszerzenie FMOD i oprawa dźwiękowa
- Tworzenie map
- Rysowanie części zestawów kafelków głównie w obrębie domu
- Dialog i fabuła

3. Analiza istniejących rozwiązań

Rynek gier dwuwymiarowych z gatunków gier z widokiem z boku i klasycznych gier fabularnych, prężnie rozwija się od kilkudziesięciu lat i do tej pory powstało dużo popularnych tytułów. Aby wstępnie zobrazować stan rynku, można wysłużyć się danymi przedstawionymi na stronie SteamDB [12]. Dostarczają one statystyk dotyczących aktywności graczy na platformie Steam, takich jak rekordowa liczba graczy jednocześnie w skali jednej doby lub wszechczasów.

3.1. Analiza rynku gier platformowych

Jak można zauważyć wiele gier widocznych na Rys. 3.1, powstała dzięki wysiłkom często jednego twórcy, przykładem mogą być “Celeste” [13] lub “Geometry Dash” [14]. Kilka gier z powyższej listy powstało dzięki pracy mniejszych niezależnych grup developerskich, tutaj przykładem mogą być gry “Hollow Knight” [15] albo “Blasphemous” [16].

#	SteamDB.info	Name ↑↓	Current ↑↓	24h Peak ↓↑	All-Time Peak ↓↑	↑↓
1.		Geometry Dash	8,512	20,348	88,346	+
2.		MapleStory	5,483	7,431	13,850	+
3.		Hollow Knight	3,127	6,694	20,324	+
4.		Tap Ninja	2,491	2,821	3,423	+
5.		Skul: The Hero Slayer	950	1,889	18,970	+
6.		Cuphead	777	1,685	19,026	+
7.		Celeste	721	1,675	7,782	+
8.		ENDER MAGNOLIA: Bloom in the Mist	462	1,042	14,134	+
9.		Your Only Move Is HUSTLE	303	963	3,405	+
10.		CounterSide	545	877	5,199	+
11.		ROUNDS	205	726	2,269	+
12.		PICO PARK 2	123	682	9,782	+
13.		Ori and the Blind Forest: Definitive Edition	279	666	9,079	+
14.		Blasphemous	285	639	3,247	+
15.		Prince of Persia The Lost Crown	237	560	1,446	+

Rys. 3.1. Statystyki popularności gier z widokiem z boku z ostatnich 24 godzin na Steam

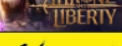
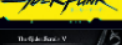
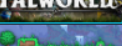
Wszystkie te gry cechuje specyficzny mechanizm nawigacji postacią przez gracza, która porusza się w przestrzeni dwuwymiarowej na przygotowanych przez twórców platformach. Do tego postać, aby progresować na mapie musi się z reguły poruszać

w prawą lub lewą stronę stąd nazwa gry widokiem z boku.

3.2. Analiza rynku gier fabularnych

Poniżej zostały przedstawione statystyki gier skategoryzowanych jako gra fabularna, na stronie SteamDB [12]. Jak wskazuje nazwa, gry te umiejscawiają gracza w świecie wykreowanym przez ich fabułę, oprawę wizualną i dźwiękową oraz mechaniki zaimplementowane w rozgrywce. Sam gracz musi się odnaleźć w tym uniwersum, mając przy tym możliwość wcielenia się w głównego bohatera.

Spośród wszystkich gier widocznych na Rys 3.2, można wyróżnić gry "Stardew Valley" [17], "Terraria" [18] i "Project Zomboid" [19], które zostały stworzone głównie przez pojedynczych deweloperów lub małe studia. Większość z pozostałych gier została stworzona przez większe studia, ale w większości wszystkie te gry dysponują bogatymi fabularnie światami, mimo, że ich podejście do wprowadzania gracza w historię uniwersum może się diametralnie różnić. Przykładowo "Elden Ring" [20] i "Terraria" [18] podają minimalną ilość informacji dla gracza, który jest zmuszony do domysłów, czytając opisy przedmiotów czy sporadyczne dialogi. Z kolei zupełnie inne podejście przedstawiają gry takie jak "Skyrim" [21], "Cyberpunk 2077" [22] czy "Kingdom Come: Deliverance II" [23], które często wyróżniają się wręcz kinową fabułą i narracją.

#	SteamDB.info	Name ↓↑	Current ↓↑	24h Peak ↓↑	All-Time Peak ↓↑	
1.		Dota 2	422,260	554,246	1,295,114	+
2.		Monster Hunter Wilds	167,956	439,862	1,384,608	+
3.		Warframe	91,285	101,881	189,837	+
4.		Baldur's Gate 3	62,959	64,491	875,343	+
5.		Stardew Valley	52,182	122,285	236,614	+
6.		Kingdom Come: Deliverance II	50,571	53,916	256,206	+
7.		ELDEN RING	43,276	58,434	953,426	+
8.		THRONE AND LIBERTY	35,045	41,200	336,300	+
9.		Cyberpunk 2077	32,446	42,919	1,054,388	+
10.		The Elder Scrolls V: Skyrim Special Edition	28,904	29,335	69,906	+
11.		Palworld	28,655	51,318	2,101,867	+
12.		Terraria	27,929	43,631	489,886	+
13.		Path of Exile 2	27,865	30,296	578,569	+
14.		Project Zomboid	26,823	27,401	65,505	+
15.		tModLoader	26,348	41,235	52,829	+

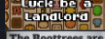
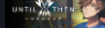
Rys. 3.2. Statystyki popularności gier z kategorii "RPG" na platformie SteamDB

3.3. Analiza rynku gier zaimplementowanych za pomocą Godot Engine

Kolejnym ważnym punktem w analizie musi być fragment traktujący o rynku gier zaimplementowanych przy pomocy silnika Godot [6]. Czyli dosyć popularnego silnika szczególnie wśród niezależnych twórców, którzy nie przynależą do większych grup deweloperów gier. Raczej nie ma na rynku tytułów reprezentujących ten silnik, które pochodziłyby od większych twórców. Wynika to z dużo większej społeczności

w przypadku silników takich jak Unity [24] i Unreal [25] oraz ich oficjalnego wsparcia dla portów konsolowych. Niestety kwestia konsol jest nieco bardziej skomplikowana w przypadku Godot [6] z uwagi na jego otwarto-źródłową naturę. Biorąc to pod uwagę imponujący jest wynik dwóch tysięcy gier zaimplementowanych w tym silniku według baz danych platformy SteamDB [12].

Jak można zauważyć każda z gier, uwzględnionych w statystyce z rysunku 3.3, miała swoją premierę w ciągu ostatnich dwóch lat, co świadczy o relatywnie niedawnym wybuchu popularności tego silnika wśród deweloperów. Jedynym wyjątkiem jest gra Virtual Cottage [26], która oficjalnie zadebiutowała pięć lat temu. Dla nas najważniejszą informacją jest to, że większość tych gier to w gry dwuwymiarowe, co świadczy o świetnym przystosowaniu silnika do tworzenia tego typu produktów.

↑↓	Name ↑↓	% ↑↓	Price ↑↓	☆ Rating ↑↓	📅 Release ↑↓	👤 Follows ↑↓	👤 Online ↑↓	📊 Peak ↑↓
	 Brotato	-40%	13,79zł	95.23%	Jun 2023	73,986	6,192	38,905
	 Tiny Pasture		14,99zł	89.99%	Feb 2025	10,950	4,575	19,732
	 Unnamed Space Idle		Free	89.03%	Jul 2023	2,111	4,223	6,385
	 Backpack Battles	-10%	53,99zł	88.59%	Mar 2024	48,670	2,164	36,557
	 Feed the Cups	-10%	65,69zł	30.00%	Feb 2024	13,732	1,296	11,413
	 That's not my Neighbor		9,99zł	79.57%	Mar 2025	4,573	662	2,975
	 Chillquarium	-30%	19,25zł	89.91%	Sep 2023	7,607	643	7,812
	 Dome Keeper	-60%	27,99zł	89.86%	Sep 2022	67,851	640	6,883
	 Halls of Torment	-25%	14,99zł	93.76%	Sep 2024	31,757	428	26,061
	 Nine-Ball Roulette	-20%	14,39zł	79.52%	Jan 2025	2,213	362	2,146
	 Your Only Move is HUSTLE	-20%	18,39zł	94.28%	Feb 2023	30,604	356	3,405
	 Virtual Cottage		Free	94.85%	Oct 2020	12,930	278	1,801
	 Luck be a Landlord	-50%	22,99zł	90.79%	Jan 2023	12,011	241	2,544
	 The Roottrees are Dead	-25%	61,49zł	92.55%	Jan 2025	7,985	240	1,116
	 Until Then	-40%	55,19zł	94.49%	Jun 2024	25,084	195	639

Rys. 3.3. Statystyki popularności gier zaimplementowanych przy pomocy silnika Godot

3.4 Analiza rynku na przykładzie wybranych gier

3.4.1. *Earthbound*

Znane w Japonii jako Mother 2 jest to część kultowej serii wydanej przez Ape i HAL Laboratory 27 sierpnia 1994 roku na Super Nintendo opowiadające o przygodach drużyny dzieci podróżującej po całym świecie próbując pokonać wielkie zło. Gra wykorzystuje dwuwymiarową mapę do poruszania się po całym świecie i system RPG do walki i nadzoru postaci. W mapie twoja drużyna wchodzi w interakcje z różnymi postaciami i rzeczami, żeby albo przejść do dalszego etapu, zdobyć przedmiot lub kupić go w sklepie, czy też rozpocząć walkę z przeciwnikiem. Walka w grze jest turowa i przedstawia przeciwnika w psychodelicznym tle, podczas gdy decyzje gracz podejmuje w ramce powyżej, a na dole znajdują się ramki ze statusem postaci. Pomimo tego wszystkiego część, która wyróżnia tą grę od reszty jest historia opowiadająca o trudach dorastania, absurdu dorosłego życia oraz pięknie przyjaźni i miłości. Gra pomimo fatalnych sprzedaży poza Japonią, zyskała natomiast gorliwą społeczność fanów, która inspirując się historią i mechaniką tej gry stworzyła wiele modyfikacji do samego “Earthbound”, a nawet niezależne gry, które osiągnęły swój własny sukces jak na przykład “Undertale” [27].



Rys. 3.4. Zrzut ekranu z rozgrywki gry “EarthBound”

3.4.2. *Fear and Hunger*

Wydana 11 grudnia 2018 przez studio Happy Paintings "Fear and Hunger" jest grą dwuwymiarową z gatunku horror posiadającą elementy fabularne. Cała gra włącznie z muzyką i portretami postaci została wykonana przez Miro Haverinen w silniku RPG Maker MV [28]. Historia tej gry zaczyna się przed wejściem do lochów, gdzie jedna z czterech wybranych postaci ma zamiar wejść w poszukiwaniu przywódcy najemników o imieniu Legarde. Rozgrywka opiera się głównie na przetrwaniu, gdzie surowców jest mało i jest szansa, że ich nie znajdziesz, walka z przeciwnikami jest turowa, ale w większości przypadków przeciwnicy są o wiele potężniejsi od głównej postaci, więc najlepiej unikać walki, a lochy są rozsiane pułapkami, które mogą z łatwością zabić cię. Pomimo mrocznej atmosfery "Fear and Hunger" wyróżnia się poprzez bardzo rozbudowane mechaniki, system walki mający wpływ na postać nawet po pokonaniu przeciwnika i fabułę przedstawianą w drobnych ilościach, która ostatecznie kumulują się w zakończeniach każdej z postaci. Wszystko to sprawiło, że gra miało bardzo dobre wyniki sprzedaży na platformie "Steam" [29], a nawet doczekała się kontynuacji w postaci "Fear and Hunger 2: Termina" [30].



Rys. 3.5. Zrzut ekranu z walki w grze "Fear and Hunger"

3.4.3. *Sally Face*

Gra zaprojektowana przez Steve'a Gabry, również znanego jako Portable Moose, w silniku Unity "Sally Face" [31] jest grą przygodową z elementami fabularnymi. Cała rozgrywka opiera się na poruszaniu się lub interakcji z postaciami poprzez kliknięcie w dane miejsce oraz rozwiązywaniu łamigłówek, których część z nich jest rozsydana po różnych rozdziałach tej gry. Pomimo swej prostoty gra ma na celu opowiedzenie historii o grupie przyjaciół, którzy poprzez swoją przyjaźń i zaufanie próbują przeciwstawić się satanistycznemu kultowi zmieniający skóry. Dzięki wcześniej wspomnianym aspektom "Sally Face" [31] stał się sukcesem wśród gier tworzonych przez małe studia, a nawet doczekał się portów na konsole Nintendo Switch [32], Xbox One [33] oraz Playstation 4 [34].



Rys. 3.6. Zrzut ekranu z gry "Sally Face"

4. Opis wymagań systemu

4.1. Wymagania niefunkcjonalne

- Gra ma zostać zaimplementowana przy użyciu silnika Godot [6]
- Spełnienie minimalnych i maksymalnych założeń wymagań sprzętowych powinno gwarantować płynne działanie przy odpowiednio minimalnych i maksymalnych ustawieniach graficznych
- Gra ma gwarantować działanie na systemie operacyjnym Windows [35] lub Linux [36]
- Interfejs gry powinien umożliwiać intuicyjne nawigowanie między widokami w trakcie gry
- Sterowanie postaci gracza w trakcie ruchu musi być w pełni responsywne na sygnały sterowania. W taki sposób, że może ona płynnie przechodzić między stanami ruchu, takimi jak “skok”, “bieg” i tym podobne
- Design gry ma w pełni uzupełniać jej historię będąc przy tym w spójny i ma on obejmować dźwięk, oprawę wizualną i rozgrywkę
- Rozgrywka ma mieć zbalansowany okienkowy system walki w stylu klasycznych gier fabularnych

4.2. Wymagania funkcjonalne

- Postaci wraz z powiązanymi z nimi mechanikami
- Map dwuwymiarowych do poruszania się po lokacji
- Systemu walki turowej i statusu postaci
- Dialogu z postaciami niebędącymi graczem i interakcji z przedmiotami w grze
- Ekranu menu i zakończeń
- Maszyny stanów postaci gracza
- Menadżera dialogów
- Obsługi plików zapisu gry
- Interfejsu gracza
- Menadżera scen
- Addona FMOD [37]
- Menu ekwipunku
- Elementów ekwipunku
- Designu postaci gracza, postaci innymi niż gracz i przeciwników

- Oprawy wizualnej
- Oprawy dźwiękowej

4.3. Założenia gry

4.3.1. Poziomy w grze:

- Gra ładuje poziomy z zasobów za każdym razem, kiedy gracz wchodzi do nowej lokacji lub kiedy uruchamia zapisaną grę
- Pokazuje miejsca, które są pełne przeciwników i przyjaznych postaci do interakcji lub są specjalne zrobione do zapisania gry.
- Automatycznie generują przeciwników i postacie w określonym miejscu.
- Posiadają miejsca na mapie, z którymi można wchodzić w interakcje.
- W niektórych lokacjach można napotkać, specjalnych przeciwników, których trzeba pokonać, jeśli gracz chce zdobyć jedno z zakończeń.

4.3.2. Postacie:

- Można je znaleźć w różnych miejscach na mapie
- Jest możliwe wejście w wachlarz interakcji pokroju dialogu czy ataku.

4.3.3. Dialog:

- Posiada wiele różnorodnych opcji.
- Niektóre opcje są odblokowane, kiedy odpowiednie wymagania zostały spełnione.

4.3.4. Przeciwnicy:

- Wejście w interakcje z nimi rozpoczyna walkę
- Atakują i osłabiają postać gracza
- Po pokonaniu zostają usunięci z poziomu

4.3.5. Walka:

- Jest oparta na systemie tur
- Gracz posiada cztery opcje: atak, magia, przedmiot i ucieczka
- Podczas walki można nałożyć na gracza i przeciwnika efekty, które albo osłabiają, albo wzmacniają ich.
- Kiedy punkty życia gracza są równe zero gra się kończy.
- Po pokonaniu przeciwnika postać gracza dostaje punkty doświadczenia lub potencjalnie przedmiot.

4.3.6. System punktacji:

- Po zdobyciu dostatecznej liczby punktów doświadczenia postać gracza osiąga nowy poziom
- Przy zdobyciu nowego poziomu gracz zdobywa nową umiejętność.

4.3.7. Zakończenia:

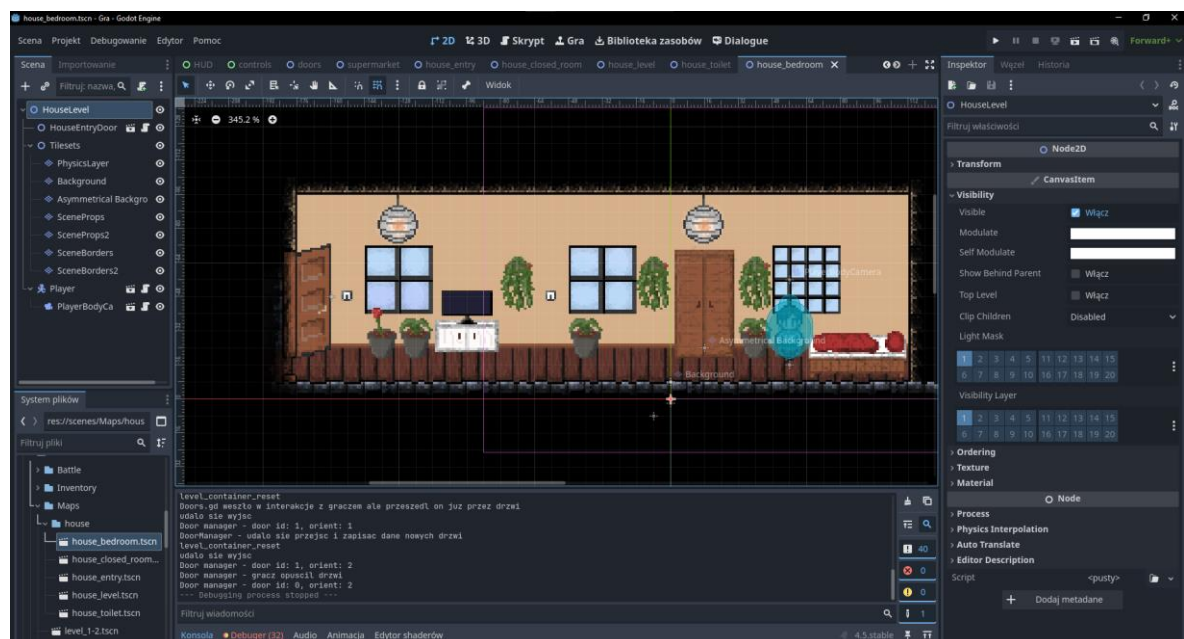
- Gra posiada dwa różne zakończenia.
- Aby zdobyć jedno z tych zakończeń trzeba spełnić określone wymagania

5. Narzędzia zastosowane w projekcie

5.1. Silnik Godot

Najważniejsze narzędzie do tworzenia gier komputerowych, czyli silnik pozwala na tworzenie projektu w swoim własnym stylu. Wiele węzłów zawartych w tym programie pozwala na umieszczenie różnych elementów niezbędnych w grze, a skrypty mogą zmieniać lub tworzyć nowe funkcje w środowisku. Nie można też zapomnieć o bibliotece zasobów zawierającej gotowe funkcje i modyfikacje pozwalające na ułatwienie pracy nad określonym etapem gry. Cały kod gry jest napisany w języku GDScript [7], który był inspirowany Python'em [38], natomiast dla pozostałych osób istnieje alternatywa w formie C# [9].

Jak widać na rysunku o numerze 5.1 jest to bardzo zaawansowany program pozwalający na pracę na własnych scenach i węzłach posiadających ogromne ilości właściwości, których wartości można edytować w inspektorze w oknie na prawo. Oferuje on możliwość ich edycji za pomocą trybu dwuwymiarowego lub trójwymiarowego oraz w samym skrypcie. Program pozwala na testowanie własnej gry za pomocą opcji uruchomienia projektu, a nawet wyeksportowanie go do pliku wykonywalnego. Ewentualne błędy lub komunikaty otrzymywane od aplikacji można monitorować w sekcji konsoli na dole. Zapewniony jest sprawny dostęp do plików projektu za pomocą okien systemu plików i sceny oraz możliwe jest wybranie własnego edytora tekstowego w opcjach projektu.



Rys 5.1. Zrzut ekranu przedstawiający interfejs Godot Engine

5.2. Język programistyczny GDScript

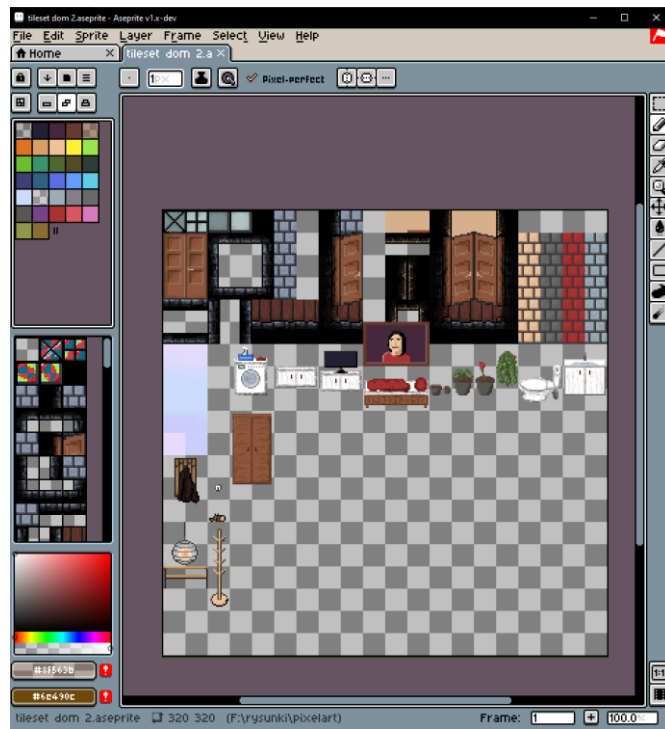
Ważnym ogniwem silnika Godot [6] jest dedykowany język GDScript [7], który został już wspomniany w podrozdziale 5.1. Jest to wysoko poziomowy, obiektowy język, którego składnia była mocno inspirowana językiem Python [38]. Wszystko to oznacza, że jest przyjazny dla początkujących chcących się nauczyć podstaw programowania i jednocześnie doskonale wywiązuje się z potrzeb silnika graficznego jakim jest Godot [6]. Stworzenie przez twórców Godot'a [6] własnego języka pozwoliło na ograniczenie złożoności kodu bez zbytejnej utraty optymalizacji, co zdecydowanie jest plusem dla nowicjuszy.

Z tego powodu jest to najpopularniejsza opcja do programowania w tym silniku. Dlatego nie powinno dziwić, że pomimo, że przewidziane jest pisanie kodu GDScript [7] głównie w natywnym edytorze silnika Godot [6], to nadal istnieje alternatywa używania innych rozwiązań takich jak Visual Studio Code [39]. Przy używania tego edytora, można nawet używać specjalnych rozszerzeń znacznie ułatwiających pracę.

5.3. Aseprite Studio

Jest to edytor graficzny do tworzenia grafiki dwuwymiarowej i idealne narzędzie do tworzenia nowych elementów graficznych w grze, licząc od postaci i wrogów po elementów zawartych w scenach map. Jest to program zapewniający rozmaite funkcje obsługujące kreację animacji, jak i obrazków w pikselowym stylu. Pozwala on na ich edycję z podziałem na warstwy, z uwzględnieniem wybranej przez użytkownika palety kolorów.

Rysunek o numerze 5.3 przedstawia narysowany własnoręcznie przez autorów zestaw kafelków używanych do tworzenia mapy domu protagonisty w grze. Program Aseprite [40], pozwala na zaawansowane opcje edycji grafiki tworzonej przez użytkownika. Posiada możliwość korzystania z palety kolorów, zaznaczeń czy narzędzi malowania, wszystko to z wysokim poziomem możliwości zmiany właściwości wybranych narzędzi.



Rys 5.2. Zrzut ekranu przedstawiający interfejs Aseprite

6. Fabuła gry

6.1. Wstępne przedstawienie fabuły i uniwersum gry

Historia gry skupia się na przygodach zwykłego pracownika biurowego zmagającego się z codziennymi wyzwaniami i presją jaką wywiera jego życie, aby po trzech dniach przejść na długo zasłużony urlop. Świat gry jest alternatywnym odpowiednikiem świata w XXI wieku, w którym jedyną różnicą są nazwy popularnych marek, ale jest za to mocno wzbogacony jego własną wyobraźnią. Z tego powodu z punktu widzenia gracza wiele elementów uniwersum ma wręcz surrealistyczny charakter, będący abstrakcyjnym odbiciem rzeczywistości.

6.2. Akt I - A

Jest tyle rzeczy, które może pójść nie tak w trakcie jednego dnia pracy. Główny bohater budzi się z tą myślą wspominając o tym, że zostały jeszcze trzy dni, do zasłużonego relaksu. Wychodząc do pracy protagonista zastanawia się w jaki sposób świat spróbuje go pokonać. Przechodząc przez ulicę coś wewnątrz niego pęka i nagle cały świat zaczyna się rozpadać, a ludzie i przedmioty zmieniają się w potwory. Po ich pokonaniu i dotarciu do wieżowca nasza postać wchodzi przez wejście i zauważa, że wszystko wróciło do normy. W biurze bohater dostaje od szefa zadanie, by wymienić tusz w drukarce, pomimo przepracowania. Przechodząc obok grupy pracowników postać, słyszy pogłoskę, że Anna ostatnio mocno przeżywa wczorajsze wydarzenia nie precyzując co się dokładnie stało. W magazynie niestety nie znajduje żadnego tuszu, więc teraz musi pójść do supermarketu, by kupić nowy. Wychodząc zauważa, że świat znowu się zmienił w jego abstrakcyjną wersję, przez którą z trudem przechodzi dostając się do supermarketu. Tam protagonista znajduje alejkę z urządzeniami biurowymi, w której zaczepia go pracowniczka próbująca zmusić go do zakupu innych produktów. Nagle zmienia się w potwora, który stoi teraz na drodze do tuszu, zmuszając tym samym pracownika do walki. Po pokonaniu jej bohater kupuje tusz, wracając tym samym do biura, gdzie montuje go w drukarce. Pod wieczór protagonista wraca do domu słysząc tym samym o tym, że Anna musiała wyjść przedwcześnie z pracy. Ignorując swoich rówieśników protagonista wraca do domu, przy czym kładzie się do łóżka kończąc tym samym dzień w mieście o nazwie "Raj".

6.3. Akt I-B

Zamiast pójść do magazynu w poszukiwaniu tuszu bohater decyduje się pójść do Anny porozmawiać z nią, o tym co się stało. Spotykając ją przed biurowcem Anna tłumaczy, że wczoraj jej chłopak rzucił ją przed jej rodzicami i trudno jest jej się pogodzić z tym faktem. Próbując poprawić jej humor protagonista zaprasza ją na lody do pobliskiego sklepu,

lecz na jego drodze stoją przeciwnicy z poranka, z którymi szybko się rozprawia. Wchodząc do sklepu Ania pyta się bohatera, dlaczego podczas spaceru co jakiś czas zatrzymywał się na środku chodnika, ale on odpowiada, że coś mu wpadło do oka. Podczas szukania lodów Ania opowiada, o tym jak chodziła ze swoim chłopakiem od jakiś trzech lat i nie może pojąć z jakiego powodu ją opuścił szczególnie po tylu latach, w których zawsze się rozumieli i starali być dla siebie. Bohater przyznaje, że sam nie rozumie, dlaczego ktoś mógł kogoś opuścić po tylu pięknych latach znajomości, ale podkreśla, że nawet jeśli jej chłopak miał jakiś powód, żeby to zrobić nie zmienia to faktu, że ostatecznie to przewyżczyło ich miłość i nie powinna zwracać sobie głowę kimś kto, by potraktował swoją dziewczynę w taki sposób. Anna dziękuje za jego słowa i para zauważając tusz leżący na półce kupuje go wraz z lodami, po czym razem wracają do drapacza chmur. Po zamontowaniu tuszu i zakończeniu pracy bohater spotyka się z Anną przed budynkiem, co skłania go do powrotu do domu z nią. Pracowniczka wyznaje protagoniście, że po dniu namysłu zdecydowała, że teraz weźmie sprawy w swoje własne ręce i postara się wrócić do normalności, przy czym wspomina, że gdyby główna postać miała kłopot pomoże mu w tym, za co protagonista dziękuje. Po pożegnaniu się z Anną i powrocie do domu, protagonista kładzie się do łóżka rozmyślając, o tym jak to ludzie mogą być lekkomyślni, jeśli chodzi o miłość i rodzinę, po czym zasypia kończąc dzień.

6.4. Akt II - A

Protagonista tak jak dzień wcześniej po obudzeniu wychodzi z domu, aby rozpocząć kolejny dzień. Podczas podróży mimowolnie rozmyśla nad czekającymi go nowymi wyzwaniami w pracy, po czym jego najbliższe otoczenie przeobraża się, w alternatywną i nieco bardziej agresywną wersję. Po rozprawieniu się z potworami świat wraca do normy, a główny bohater dociera do firmy, w której pracuje. Spotyka tam swojego ambitnego współpracownika, znanego z bycia przykładnym, ale często podlizującym się osobom wyżej, kolegą z roboty. Po krótkiej rozmowie protagonista przechodzi dalej i dostaje polecenie, aby dostarczyć dokumenty do szefa, który przebywa w pobliskiej kawiarni. Prezes ruga go krytykując spóźniałstwo i nastawienie do pracy, a po dłuższym momencie takiego monologu przeobraża się w kolejnego potwora, atakującego głównego bohatera. Po pokonaniu prezesa wszystko wraca do normy, a protagonista powraca do swojej zwykłej biurowej pracy. Dalszy dzień mija w normalny sposób, ale główny bohater jest w stanie przez niego przebrnąć, myśląc o zbliżającym się urlopie. Po powrocie do domu słyszy intrygującą historię, ale jest na tyle zmęczony, że od razu kieruje się do swojego łóżka, aby spać.

6.5. Akt II - B

Dzień mija w analogiczny sposób jak wcześniej, ale różnice zaczynają się przy interakcji z ambitnym współpracownikiem nazywającym się Robert. Główny bohater po spotkaniu go w miejscu pracy, tłumaczy mu swoją sytuację i nawiązują wspólną nić zrozumienia. Współpracownik opowiada o swojej przeszłości, kiedy włożył ogromne ilości wysiłku, aby znaleźć się w miejscu i firmie, w których obecnie się znajduje. Musiał on spełnić wymagania swojej własnej rodziny i otoczenia, co nieustannie pchało go do parcia dalej w trakcie edukacji i później wspinania się po korporacyjnej drabinie. Zauważa, że nie chce zaprzepaścić swoich własnych wysiłków, co zmusza go do tytanicznej pracy na rzecz firmy i często innych współpracowników. Robert po rozmowie z protagonistą dochodzi do wniosku, że ciągle wysilanie się pod presją otoczenia nie jest zdrowe i pyta się głównego bohatera czemu często stoi jak słup soli z pustym wzrokiem bez żadnego kontaktu. Dostaje odpowiedź, że wynika to z potrzeby chwilowego oderwania się od rzeczywistości, choć często taka chwila przedłuża się. Po rozmowie kierują się do biura dyrektora, a wizyta u prezesa, jak i reszta dnia przebiega podobnie jak pierwotnie z tą różnicą, że dyrektor zauważa wysiłki Roberta i wyznacza mu urlop, który ma mu pomóc w przepracowaniu.

6.6. Akt III - A

Następnego dnia przychodząc do pracy protagonista dostaje najtrudniejsze zadanie, jakie kiedykolwiek istniało, przejście przez górę papierkowej roboty. Niestety dokumentów jest tak dużo, że bohater nie mogąc znieść nagromadzonego stresu traci przytomność. Budząc się ze snu główny bohater znajduje się teraz w fantastycznym świecie zrobionym całkowicie z dokumentów, gdzie potwory zrobione z przyrządów biurowych czyhają na jego życie. Dostając się na sam szczyt Góry Mache protagonista spotyka Lorda Origami, który śmiejąc się ujawnia głównemu bohaterowi swój nieczyny plan pokrycia całego świata makulaturą, co motywuje bohatera do walki z nim. Po wygranej walce cały świat zaczyna się ściemniać i trząść, podczas gdy głos zaczyna robić się coraz głośniejszy. Budząc się ze snu protagonista dowiaduje się od kolegi, że przez cały dzień przeglądał dokumenty z takim zapalem, jakby cały świat dla niego przestał istnieć, a po skończeniu położył się na biurku i zasnął. Wracając do domu bohater zauważa, że żaden potwór czy też alternatywna rzeczywistość się nie pojawia, z czego wnioskuje, że pokonanie Lorda Origami nareszcie zaprzestało jego halucynacje. W domu bohater kładzie się do łóżka rozmyślając, o tym co ciekawego zrobi podczas urlopu.

6.7. Akt III-B

Następnego dnia w biurze, bohater staje przed swoim największym wyzwaniem górą

papierkowej roboty. W tym momencie protagonista jest gotowy wrzasnąć nad absurdem tego zadania, ale zanim to zrobił Ania i Robert przychodzą do jego biura mówiąc, że pomogą mu z tym. Podczas pracy nad dokumentami bohater wyznaje, przed swoimi rówieśnikami, że cała ta praca i stres związany z nią nie pozwala mu się zrelaksować ani na sekundę. Robert odpowiada, że rozumie przez co przechodzi i właśnie dlatego porozmawiał z Anną, żeby mu jakoś z tym pomóc. Po skończeniu pracy na dzień dzisiejszy protagonista wspomina, że jutro jedzie na urlop, tylko nie ma za bardzo planów jak go spędzić. Anna proponuje, żeby razem pojechali nad jezioro, gdzie mogą się zrelaksować, pływać kajakami i łowić ryby na co reszta się zgadza. Wychodząc z biurowca główny bohater rozmyśla nad faktem, że teraz ma kogoś do towarzystwa podczas urlopu, co według niego będzie ciekawym doświadczeniem, po czym odchodzi w stronę słońca, mając nadzieję, że jutro będzie lepsze.

6.8. Zakończenia

Każda gra fabularna musi posiadać jakieś zakończenie, a gra omawiana w tym dokumencie ma ich aż trzy. Każde z nich można poznać przechodząc grę w określony sposób i spełniając określone warunki. W rozdziale numer sześć autorzy opisali już w zwarty sposób fabułę zakończeń A i B, oprócz tego gra ma ukryte zakończenie.

Zgodnie z diagramem w rozdziale ósmym, aby odblokować ukryte zakończenie należy pozyskać specjalny przedmiot i użyć go w określony sposób oraz zakończyć trzeci dzień w obrębie gry.

Natomiast warianty A i B można poznać po prostu przechodząc grę i kończąc trzeci dzień pracy, przy czym relacje z postaciami pobocznymi historii mają tutaj decydujące znaczenie, które zakończenie z tych dwóch to będzie. Wśród osób z którymi możliwe jest zawarcie i pogłębienie znajomości, przynajmniej w obrębie mechanik gry, można wyróżnić Anię i Roberta, którzy są kluczowymi postaciami odpowiednio w pierwszym i drugim dniu wewnątrz gry. Mechanika gry decydująca jakie zakończenie otrzyma gracz polega na inkrementowaniu wartości "przyjaźni" protagonisty w relacji z odpowiednią postacią, każdorazowo przy wybraniu odpowiedniej opcji dialogowej.

Zakończenie A

Po wielu latach uciążliwej pracy protagonista nareszcie może się zrelaksować na plaży. Leżąc na leżaku rozmyśla, że było warto przechodzić przez ten cały stres i irytujących pracowników tylko po to by mieć cały miesiąc wyłącznie dla siebie.

Zakończenie B

Protagonista siedzi sobie nad brzegiem jeziora i spogląda na spławik czekając, aż

jakaś ryba się skusi na przynętę. Tym samym zwraca swój wzrok do swoich nowych przyjaciół myśląc, że nigdy by sobie nie wyobrażał tak relaksującego urlopu i jest wdzięczny za to co jego współpracownicy wymyślili. Ma również nadzieje, że w najbliższym czasie się spotkają na więcej ciekawych przygód.

7. Opis postaci

7.1. Protagonista

Jest to główny bohater gry i postać, którą sterujemy podczas rozgrywki. Ma czarne krótkie włosy i ciemno-brązowe oczy, jeśli chodzi o budowę ciała jest chudy i wysoki. Nosi białą koszulę wraz z niebieskim krawatem, czarnym garniturem oraz czarnymi pantoflami mającymi prezentować wygląd pracownika wielkiej korporacji. Z bliska natomiast można dostrzec, że te ubrania zostały nałożone na ostatnią chwilę, sprawiając wrażenie jakby mężczyzna się gdzieś spieszył i nałożył pierwsze lepsze ubranie.

Pomimo tego, że wygląda na opanowanego, to w rzeczywistości jest człowiekiem żyjącym w ciągłym stresie. Ciągłe uciążliwe zadania sprawiają, że bohater stoi na granicy z psychozą, a to idealnie jest reprezentowane poprzez schizofrenie pokazującą mu potwory na ulicy i unoszące się kawałki ziemi. Pomimo trudności, przez które przechodzi protagonista jest zdeterminowany, by wykonać każde zadanie jakie napotka. W obu ścieżkach można też zauważyć, że podtrzymuje się w nadziei na lepsze jutro. W pierwszej ścieżce ma nadzieję na urlop, w którym zrelaksuje się jak nigdy dotąd, a w drugiej ścieżce podtrzymuje wiarę w rówieśnikach, którzy zawsze będą przy nim. Jest również głębokim myślicielem, który codziennie się zastanawia nad tym co przeżył i co to oznacza dla niego i innych.



Rys. 7.1. Zrzut ekranu na postać gracza.

7.2. Ania

Ania jest pracowniczką w dziale zarządzania zasobami ludzkimi w firmie, w której

pracuje główny bohater. Jest ona kobietą o nieskazitelnej cerze i bladej skórze. Ma długie blond włosy związane w kucyk i niebieskie oczy. Nosi fioletowy garnitur, pod którym widać białą koszulę z pomarańczowym krawatem oraz czarne buty służbowe. Wygląd ten ma pokazać, ciepłą i zrelaksowaną naturę, pod którą widać jednak profesjonalizm. Postać ta została opisana na podstawie rysunku 7.2.

Kiedy się ją bliżej pozna można zobaczyć, że jest pewną siebie osobą, która swoim zachowaniem inspirowała innych do działania. Posiada naturalne atrybuty przywódcy, które pomagają jej poprzez staranne rozplanowanie działania i przydzielenie odpowiednich ludzi do odpowiednich stanowisk tak, żeby całość szła jak najszybciej i efektywniej. Wszystko jednak się zmienia, kiedy jej chłopak rzucił ją z nieznanego powodu, co doprowadziło dziewczynę do rozpacz. W normalnej ścieżce, kobieta wraca do domu w środku pierwszego dnia i więcej jej nie widzimy, natomiast w pobocznej ścieżce protagonista zabiera ją do supermarketu, by wypełnić powierzone mu zadanie. Tego samego dnia doradza jej, żeby nie przejmowała się, tym co się wydarzyło i patrzyła w przeszłość za co mu dziękuję. W trzecim dniu pomaga protagoniście z papierkową robotą i wspiera go, kiedy on wyznaje jej i Robertowi jak się czuje. Jest ona idealnym przykładem tego jak pewność siebie i empatia sprawiają, że jest się uważanym za wspaniałą osobę i świetną liderkę.



Rys. 7.2. Zrzut ekranu na postać Ani.

7.3. Robert

Robert czyli postać, którą spotykamy w drugim dniu rozgrywki na pierwszy rzut oka

wygląda jak nienaganny pracownik o przeciętnej budowie ciała. Ubrany jest

w elegancki jaseitowy garnitur, spodnie do pary i ciemne buty, a ubiór ten uzupełnia czerwony krawat. Posiada krótko ustrzyżone włosy czarnego koloru, ciemną skórę oraz zielone oczy.

Jako istotną dla fabuły gry cechę jego charakteru można uwzględnić ponadprzeciętną ambitność i wytrwałość w dążeniu do celu. Sprawia wrażenie osoby w pełni stonowanej, potrafiącej trzymać uczucia na wodzy w sytuacjach, które tego wymagają. Nie mówi dużo, ale za to dużo robi, na przykład pomagając swoim znajomym z pracy, czym zaskarbił sobie sympatię wielu z nich. W głębi duszy żyje jednak w coraz większej niepewności, zastanawiając się nad sensem niekończących się syzyfowych wysiłków w trakcie pracy. Powieli schematy wyuczone w nim przez jego środowisko, w szczególności rodzinę w dzieciństwie, które zmuszają go do ciągłego parcia na przód kosztem własnego stanu psychicznego i fizycznego.



Rys. 7.3. Zrzut ekranu na postać Roberta.

7.4. Dyrektor Generalny

Prezes to zwyczajnie wyglądająca osoba o blond włosach i bródce oraz

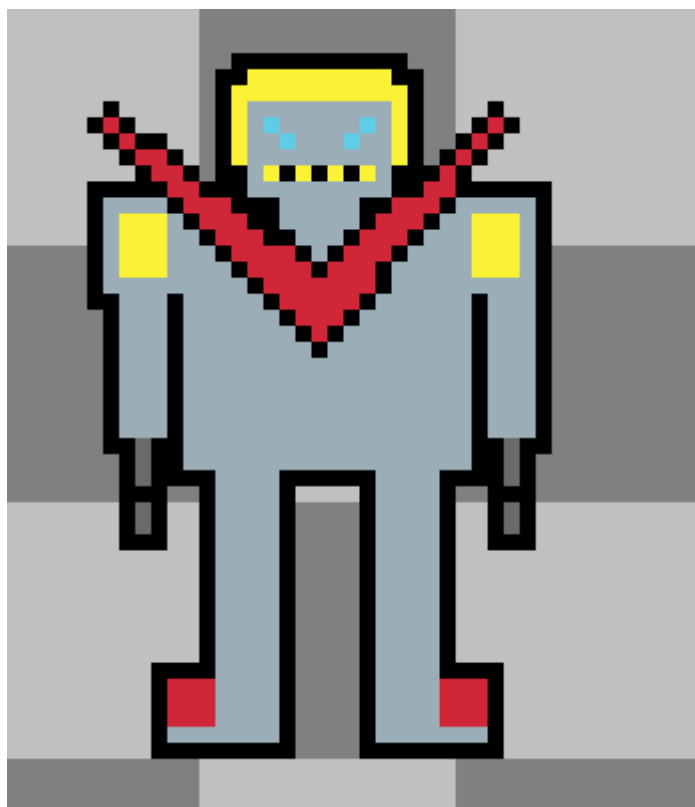
z kremowym kolorem skóry. Posiada czarne pantofle z czerwonym garniturem, białą koszulę i niebieski krawat. Wyróżniającym się elementem jego ubioru zdecydowanie jego krawat ozdobiony artystycznymi wzorami, a obrazu dopełnia modna fryzura starannie wyselekcjonowana przez sztab firmowych specjalistów. Jego wygląd jest przedstawiony na rysunku 7.4.

W trakcie napadu schizofrenii protagonisty przyjmuje on diametralnie inną postać będącą karykaturą jego normalnego wyglądu. Po przemianie wygląda jak stereotypowy robot z japońskiej animacji posiadający srebrną zbroję, czerwony znak na piersi oraz reflektory umieszczone w jego ramionach. Na jego twarzy widnieją niebieskie i żółte żarówki mające reprezentować jego oczy i usta, a jedyne co pozostało z jego oryginalnego wyglądu są jego blond włosy. Jego forma jest pokazana na rysunku 7.5.

Dyrektor jest lekko surowym, ale w gruncie rzeczy sprawiedliwym szefem i jak i człowiekiem. Wymaga od swoich pracowników w trakcie pracy tego samego co od siebie, czyli profesjonalizmu, zaangażowania i punktualności, co objawia się sporadycznymi pouczeniami. W wymagającej tego sytuacji jest w stanie jednak wykazać się empatią w stosunku do swoich podwładnych i umie docenić ich starania adekwatnie ich nagradzając.



Rys. 7.4. Zrzut ekranu przedstawiający postać szefa.

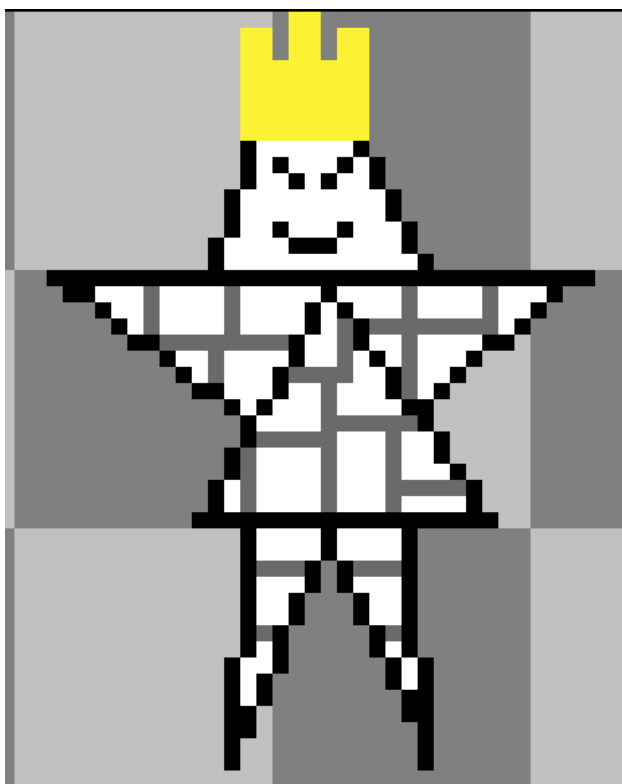


Rys. 7.5. Zrzut ekranu postaci dyrektora jako przeciwnika.

7.5. Lord Origami

Lord kształtem przypomina człowieka, ale jest coś nim nie tak, jego białe ciało składa się z origami, układającego się w całe wzory tworzące jeden organizm. Jego głowę przykrywa złota korona ułożona na jego szpiczastej głowie. Postać jest opisana na podstawie rysunku 7.6.

Pomimo zdradliwego uśmiechu na czymś co wygląda jak jego głowa, jest komicznie złą istotą, chcącą zrujnować cały świat za pomocą makulatury. Jego zachowanie w trakcie interakcji z protagonistą cechuje agresywność, poczucie wyższości i skłonność do drażnienia się, wszystko to podkreślone za pomocą kreskówkowych manieryzmów typowego antagonisty.



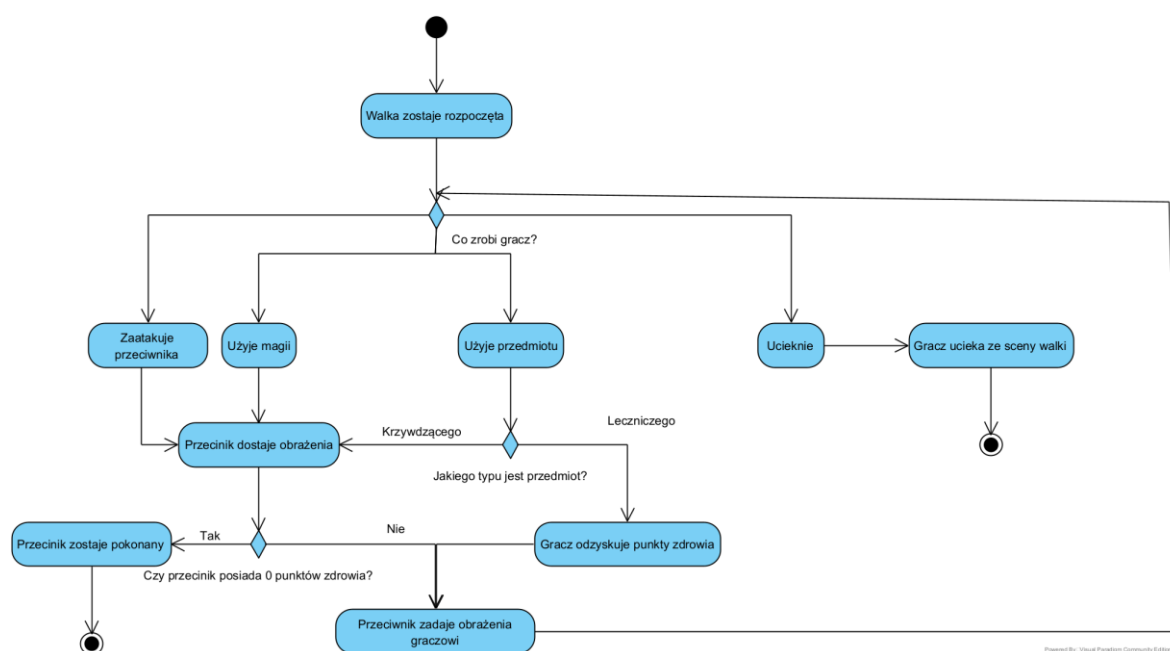
Rys. 7.6. Zrzut ekranu na postać "Lorda Origami"

8.2. Diagramy procesów

Diagram systemu walki

System turowy jest fundamentalnym aspektem walki. Na rysunku 8.3 widać, że przy rozpoczęciu postać, która jako pierwsza zaatakowała ma prawo do wykonania pierwszego ruchu. Do wybrania ma atak fizyczny, blokowanie, użycie przedmiotu oraz pominięcie swojej tury co powoduje przejście do następnej postaci w kolejce. Atak fizyczny postaci normalnie zadaje maksymalną liczbę obrażeń, jednakże bywają momenty, w których atak albo nie trafia, albo obrażenia zostają ograniczone z powodu wzniesienia gardy przez przeciwnika. Użycie przedmiotu oznacza wybór jednego z wielu zdobytych przez protagonistę elementów wyposażenia ekwipunku, których efekty są opisane w późniejszych diagramach.

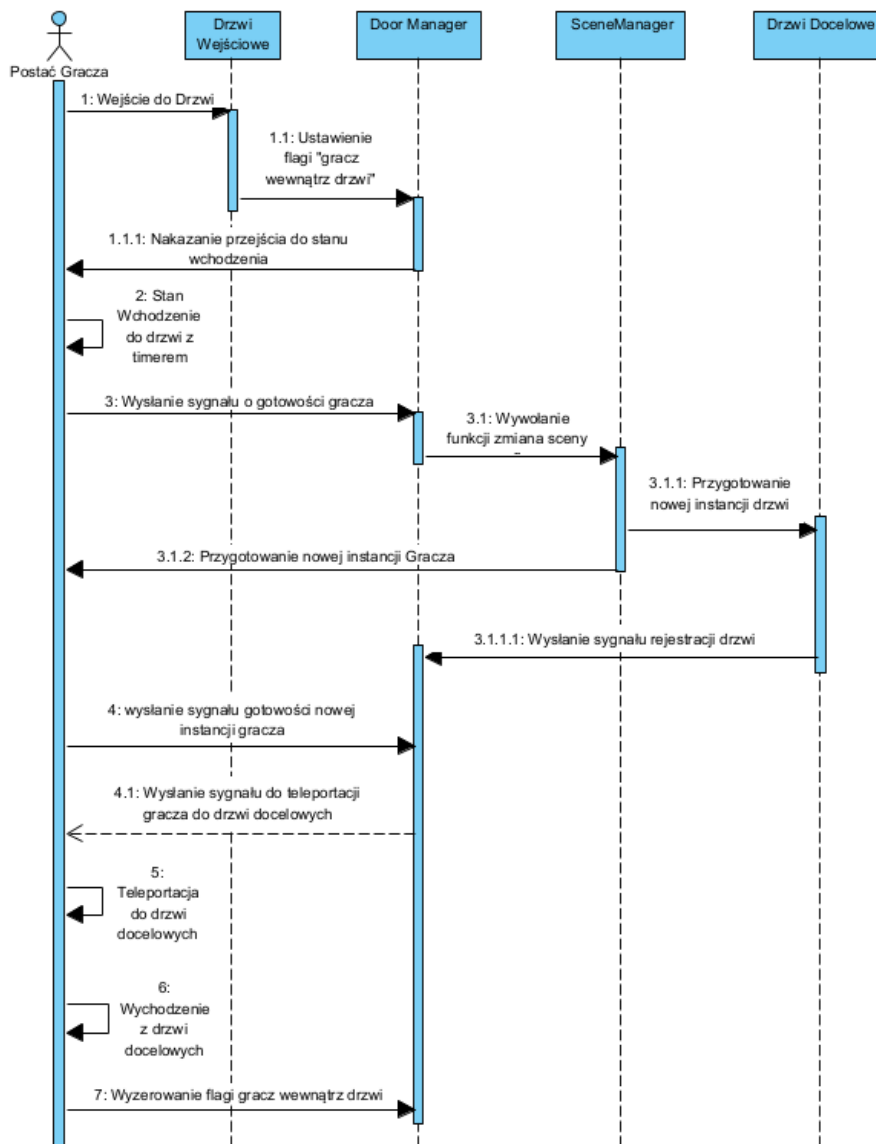
Cała walka trwa do momentu, aż wszyscy przeciwnicy zostaną pokonani lub punkty zdrowia protagonisty spadną do zera.



Rys. 8.3. Diagram systemu walki

Diagram sekwencji przechodzenia przez drzwi

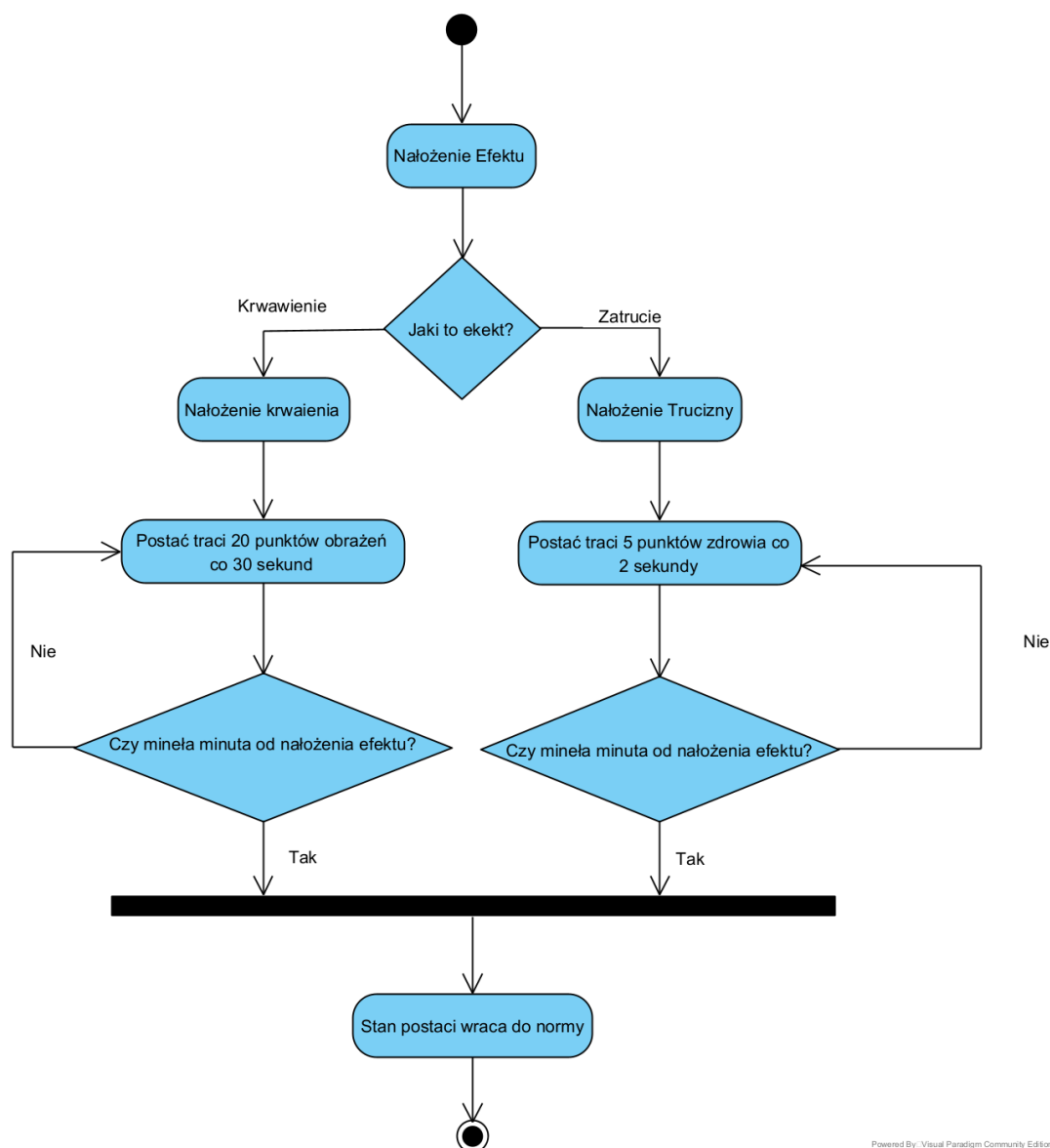
Gracz ma możliwość przejścia przez drzwi w trakcie eksploracji scen. Na potrzeby obecnego diagramu dzielą się na drzwi wejściowe i docelowe, czyli znajdujące się w obecnej scenie i docelowej. Ważnym aspektem działania całego mechanizmu jest współpraca Menedżerów Scen i Drzwi i samych w sobie instancji drzwi, między sobą za pomocą ustawiania flag i wysyłania sygnałów charakterystycznych dla języka programowania Godot. Na potrzeby utrzymania klarowności diagramu autorzy nie uwzględnili mechanizmu obsługi błędów zawartych w całym systemie. Cały proces jest przedstawiony na rysunku 8.4.



Rys. 8.4. Diagram sekwencji przechodzenia przez drzwi

Diagram aktywności nakładania efektów

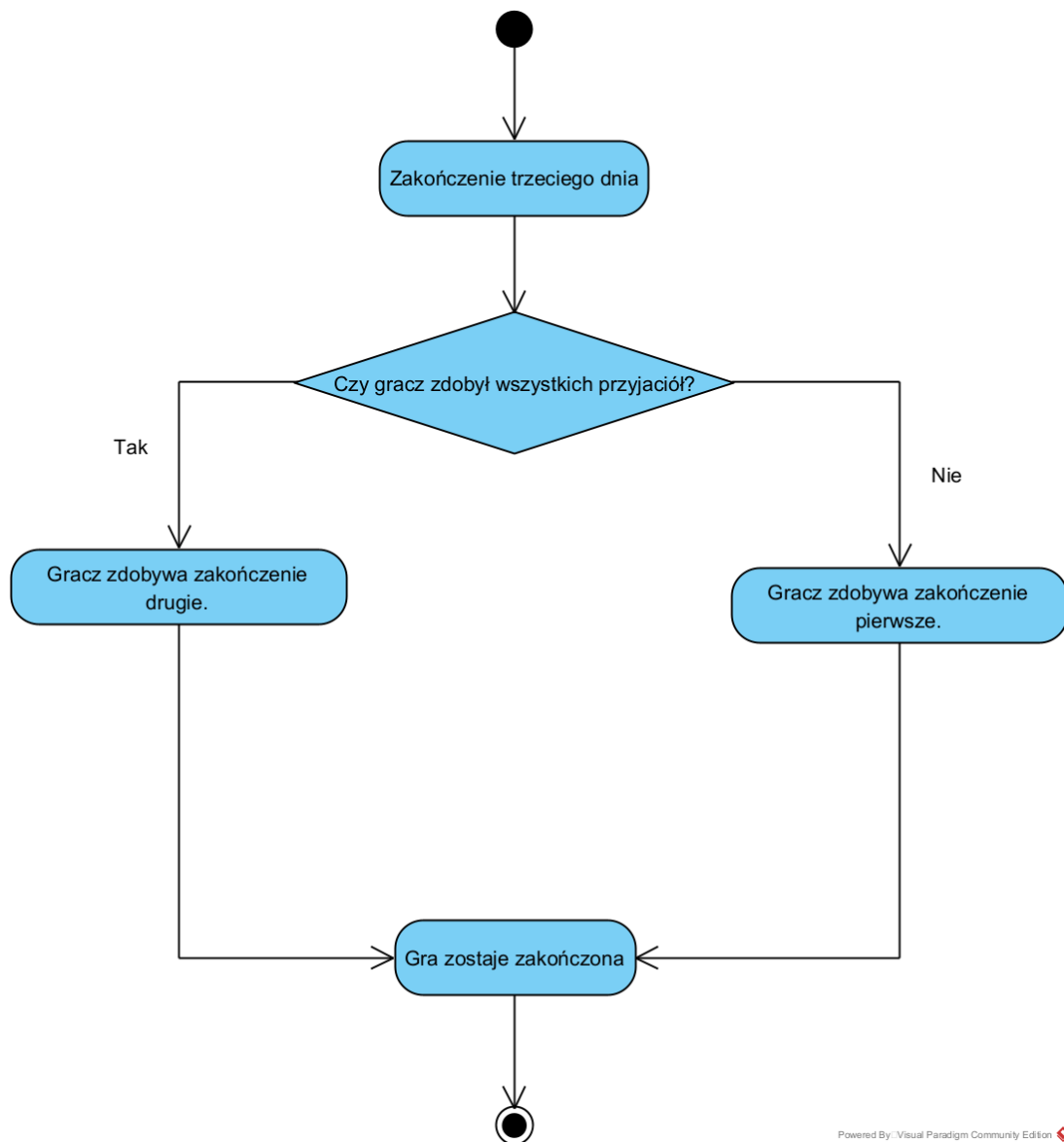
Specjalne efekty nałożone na postać są aspektem, który zwiększa trudność, każdej walki. W rysunku 8.5 widać, że status postaci można zmienić na dwa dolegliwości: krwawienie i zatrucie. Krwawienie polega na stopniowej utracie dwudziestu punktów życia co trzydzieści sekund, a zatrucie odbiera postaci pięć punktów życia co dwie sekundy. Powyższe efekty trwają minutę, po której postać wraca do normalnego stanu. Efekty dają graczowi dodatkowe wyzwanie, którym jest obmyślenie strategii opierającej się na jak najbardziej efektywnym pokonaniu przeciwnika.



Rys. 8.5. Diagram aktywności nakładania efektów

Diagram zakończeń

Zakończenia są elementem gry mającym być kulminacją wszystkiego czego gracz dokonał podczas całej rozgrywki. Z rysunku 8.6 widać, że użytkownik mógł pod koniec gry obejrzeć jedno z dwóch dostępnych zakończeń. Pierwsze zakończenie znane jako “Urlop” można zdobyć poprzez przejście trzech dni bez wykonania, żadnych dodatkowych czynności i wykonania głównego zadania. Drugie i ostatnie zakończenie, czyli “Przyjaźń to magia” jest zdobyte, tylko kiedy pod koniec gry protagonista zdobędzie wszystkie punkty od postaci, z którymi może wejść w interakcje. Wiele zakończeń ma zachęcić gracza do ponownego zagrania i odnalezienia wszystkich sekretów, które czekają na odnalezienie.



Rys. 8.6. Diagram zakończeń

8.3. Podstawowe mechaniki rozgrywki

Mapa

Mapa, czyli przestrzeń, po której gracz porusza się w trakcie rozgrywki, składa się na wiele scen, pomiędzy którymi gracz może przechodzić i je eksplorować. To na niej odbywają się interakcje z przeciwnikami, różnymi postaciami lub interaktywnymi przedmiotami.

Różnorodność poszczególnych scen zapewnia intuicyjny system kafelków, inaczej nazywany “tileset”, który pozwala na tworzenie odmiennych krajobrazów z dużą łatwością. Polega on na tworzeniu mapy, kafelek po kafelku w sposób podobny do układania puzzli, przy czym poszczególne kafelki mogą mieć różne warstwy, między innymi warstwę kolizji, czy warstwę obrazu. Popularną techniką jest również tworzenie warstwy kolizji mapy przy pomocy zestawu kafelków i przykrywanie jej za pomocą większych obrazów, których podzielenie na wiele kafelków miałooby się z celem.

System ten najczęściej uzupełniają inne elementy, na przykład poruszające się platformy, które mogą poruszać się na boki, w górę i w dół oraz umożliwiają dla gracza wskoczyć na nie od spodu, kolidując z nim jedynie od góry lub pozwalają na zeskoczenie z nich na dół po naciśnięciu odpowiedniego przycisku.

Przewiduje się również rozmieszczenie na mapie przeciwników i innych postaci, czyli obiektów z którymi gracz może wchodzić w interakcje w oknie dialogowym, charakterystycznym dla gatunku gier “RPG”. Uwzględnia się też obiekty wchodzące w interakcje z graczem po wejściu w ich kształt interakcji, przykładowo mogą odtwarzać dźwięk czy iluminować statyczne i dynamiczne obiekty wchodzące w ich zasięg.

Gracz ma możliwość opuszczenia danej sceny wchodząc w interakcje z obiektem przejścia, który może przybrać formę drzwi lub niczego – kamera w pewnym momencie przestanie się poruszać się za graczem i umożliwi mu podejście do “granicy ekranu” i dalej na sąsiednią scenę. Mówiąc o przechodzeniu między scenami, nie można pominąć stałego elementu gier “RPG”, czyli ukrytych przejść do sekretnych scen lub, co ważniejsze dla fabuły, przejść zablokowanych do momentu spełnienia określonych warunków potrzebnych do progresowania w historii. Takim warunkiem może być posiadanie określonego przedmiotu w ekwipunku czy pokonanie “bossa” czy tym podobne.

Walka

Jest to jedna z podstawowych mechanik w grze, bez której gra straciłaby zainteresowanie. Walkę rozpoczyna się poprzez interakcję z przeciwnikiem na mapie poprzez zbliżenie się do niego. Gracz po interakcji zostaje przeniesiony do sceny walki, gdzie się znajduje przeciwnik. Podczas swojej tury gracz ma do wyboru cztery opcje: atak bronią, użycie magii, użycie przedmiotu i ucieczkę. Atak zadaje obrażenia na podstawie broni jaką posiada gracz, magia pozwala graczowi rzucać zaklęcia, przycisk "Item" daje możliwość graczowi użycie przedmiotu do pomocy, a ucieczka pozwala na wyjście ze sceny bitwy. Po wykonaniu tury przeciwnik zaatakuje gracza co skraca punkty życia, jakie posiada. Cały system zachęca gracza do skorzystania z ograniczonego arsenału, by pokonać wroga. W przeciwnym wypadku, kiedy punkty zdrowia gracza osiągną zero, gra zostaje zakończona i gracz musi skorzystać z ostatniego zapisu, by wrócić do rozgrywki.

Status postaci

Każda postać w grze posiada specjalną zmienną zwaną statusem na podstawie, której mogą się wydarzyć zmiany w jej zachowaniu lub punktach zdrowia. W liście są zdefiniowane pięć wartości, które mogą być przepisane do tej zmiennej. Pierwszą z tych wartości jest "normalny", każda postać i przeciwnik posiada ten status i ma on sugerować, że nic się nie przydarzy danej postaci i w żaden sposób jego punkty nie zostaną zmienione. Drugą wartością jaką można napotkać jest "krwawienie", wyróżnia się tym, że co trzydzieści sekund postać posiadająca ten status traci dwadzieścia punktów zdrowia jakie posiada w danej chwili. Trzecią i ostatnią wartością w liście jest "zatrucie" i tak samo jak z "krwawieniem" postać z tym statusem traci punkty zdrowia, z tą różnicą, że co dwie sekundy traci pięć punktów zdrowia.

Okna interakcji z elementami mapy

Warto wyróżnić obiekty, z którymi gracz ma możliwość przeprowadzenia interakcji przy pomocy okien interfejsu, przykładowo okna dialogowego przeciwników i innych postaci oraz przedmioty rozrzucone po mapie.

W przypadku okna dialogu z postacią przyjmuje ono wygląd prostokąta, którego interakcja pokazuje tekst symbolizujący interakcję między graczem, a postacią. Czasem podczas dialogu ukazuje się lista opcji dialogowych, które gracz może wybrać, aby przejść do dalszych opcji dialogu.

Na mapach platformowych można znaleźć przedmioty, które są ukazane jako białe gwiazdy. Interakcja z nimi spowoduje dodanie nowego przedmiotu do ekwipunku, co może

być pomocne podczas walki. Wśród przedmiotów można wyróżnić surowce, które pomagają protagoniście w pojedynku i ekwipunek, który wzmacnia atak lub defensywę postaci, którą grasz.

Sterowanie

Kolejną podstawową mechaniką gry jest sterowanie, które polega na interpretowaniu przez grę sygnałów przesyłanych przez gracza. Przykładowo gracz ma możliwość naciśnięcia przycisku na klawiaturze i jeśli jest on przypisany do któregoś sygnału sterowania w grze, zostaje wykonana akcja przewidziana przy przechwyceniu tego sygnału. Istnieje możliwość zmiany przypisanych przycisków na inne, preferowane przez gracza, tak aby miał pełną swobodę w trakcie gry. Szczególnie ważnym faktem jest to, że jeden przycisk sterowania gry może mieć wiele funkcji w zależności od tego co robi gracz i gdzie się znajduje (na przykład w menu głównym lub w widoku mapy).

Można wyróżnić wiele przycisków sterowania, jednak warto przedstawić najważniejsze dla mechaniki rozgrywki. Chyba najważniejszym jest przycisk gry kojarzony z lewym przyciskiem myszki, pozwalający na przykład na wybór opcji w menu gry. Kolejne ważne przyciski, pozwalają na chód w lewą lub prawą stronę, sprint, skok, zejście z platformy, przy czym celowo nie jest podana informacja odnośnie tego jakim przyciskom na klawiaturze one odpowiadają, ponieważ są to przyciski sterowania gry, nie te które wybiera dla siebie gracz. Oprócz tego kolejnym kluczowym elementem sterowania jest sensor myszki pozwalający na szybkie nawigowanie w wielu oknach menu, jednak w razie jego braku, możliwe jest użycie przycisków chodu w lewo, prawo i tym podobne, czyli kojarzone szerzej jako “WASD” lub “strzałki”. Warto również wspomnieć o przyciskach takich jak przycisk przejścia do menu głównego (również wyjścia z okna), ekwipunku lub dziennika.

W kontekście samej rozgrywki i sterowania na mapie ważnymi przyciskami są odpowiadające za chód, po naciśnięciu któregoś z nich postać porusza się na prawo lub lewo i wyświetlana jest odpowiednia animacja (tak jak zawsze po naciśnięciu przycisku sterowania postacią), jeśli dodatkowo gracz naciśnie przycisk sprintu postać zaczyna poruszać się zauważalnie szybciej. W sytuacji, kiedy postać zejdzie z podłoża, na którym się znajduje i przestanie kolidować to fizyka gry spowoduje, że zacznie opadać, a jeśli znajduje się na podłożu i gracz naciśnie “skok” to postać podskoczy. Oprócz tego gracz znajdując się na platformie ma możliwość zeskoczenia z niej naciskając odpowiedni przyciski.

8.4. Poziomy mapy

Poziomy mapy można podzielić na te znajdujące się w rzeczywistości i te będące jej

odwzorowaniem w wyobraźni protagonisty. Z reguły mają one cechy wspólne jak ogólną podstawową strukturę i kierunek, w który powinno się kierować, aby ukończyć ten poziom. O ile pierwsza kategoria poziomów przypomina w większości współczesny świat rzeczywisty, z wieloma elementami takimi jak drogi, chodniki, budynki i ich wnętrza to zazwyczaj ma ona jedynie jedną platformę, po której postać może poruszać się w prawo lub lewo. Z kolei druga kategoria nadal zawiera te same elementy, ale zmodyfikowane w taki sposób, aby odwzorować stan psychiczny protagonisty, to może oznaczać zabawę światłem w taki sposób, że zmienia ono kolor lub staje się słabsze, a może też oznaczać użycie zmodyfikowanych, trochę bardziej surrealistycznych, wersji przedmiotów z pierwszego świata. Przy zmianie typu świata może zmienić się również struktura świata, ale zazwyczaj nie w diametralnym stopniu, przeważnie oznaczać ma to dodanie kilku znajdziek i jego poszerzenie.

Dom

Dom to jak wskazuje nazwa dom protagonisty i znajduje się w nim wszystko potrzebne do życia dla głównego bohatera gry z gatunku "RPG". Zawiera on łóżko, podłogę, drzwi, niebieskie okna, telewizor, może nawet szafkę i inne niektóre elementy wyposażenia mieszkania.

Drogi

Drogi obejmują przestrzeń od punktu startu do punktu docelowego, włącznie z widokiem na te budynki od zewnątrz. Pomiędzy nimi znajduje się droga z chodnikami, jezdnią, przejściami dla pieszych i samymi pieszymi, którzy się nie poruszają, ale da się wchodzić z nimi w interakcje.

Biuro, Kawiarnia, Supermarket

Samo wnętrze biura wygląda dokładnie tak jak wszystkie inne biura, czyli zawiera miejsca pracy z biurkami i komputerami, ksero i drukarkę, plakaty i błękitny dywan. Znajdują się w nim pomieszczenia głównego bohatera, prezesa, Roberta. Analogicznie sytuacja wygląda w kontekście Kawiarni i Supermarketu, w tym sensie, że są one klasycznym i przewidywalnym odwzorowaniem współczesnego świata w rozpikselowanym świecie.

Poziomy odzwierciedlające wyobraźnię protagonisty

Takie "Levele" wyznaczają się przede wszystkim bardziej surrealistycznym charakterem, ale nadal są rozpoznawalne jako zniekształcony rzeczywisty świat. Mają również wiele platform nie tylko jedną przeważającą, a na nich znajdują się agresywni wobec protagonisty przeciwnicy, z którymi można wejść w walkę.

Góra papierkowej roboty

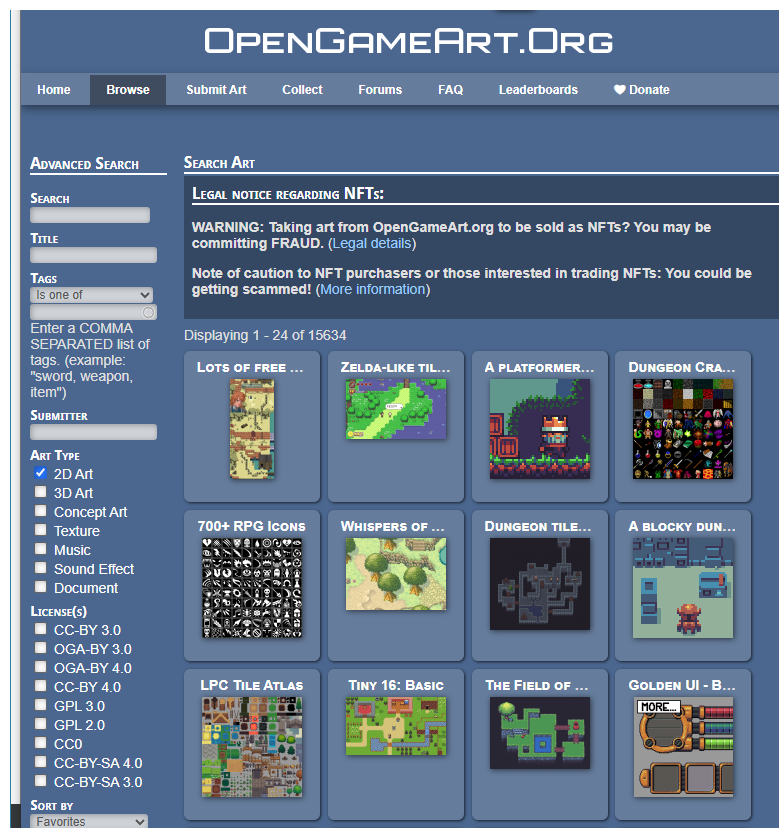
Ważnym do wspomnienia elementem jest Góra papierkowej roboty, będąca najbardziej surrealistycznym poziomem, który nie da się zasufladkować jako zwykłe odzwierciedlenie rzeczywistości. Kluczowym elementem jest sama w sobie góra papieru, na której znajduje się główny przeciwnik.

9. Elementy grafiki

Oprawa wizualna i dźwiękowa to integralne części większości gier komputerowych, dlatego ważnym jest, aby poświęcić im wystarczająco uwagi. Dużo zespołów deweloperskich o większym stopniu profesjonalizmu mają oddzielnych twórców odpowiedzialnych za te dwa aspekty pracy grupy, dlatego grafika i dźwięk w tej grze są w większości wzięte z Internetu lub utworzone przez twórców, którzy inspirowali się innymi grami.

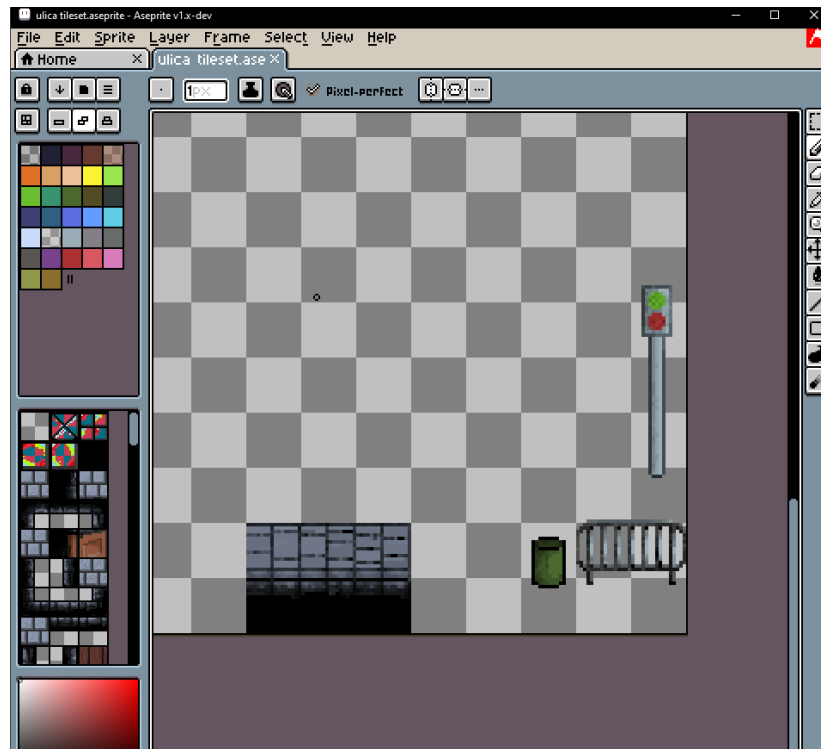
9.1. Dostępne źródła zasobów graficznych

Twórcy postanowili użyć darmowych zasobów dostępnych w Internecie, na potrzeby tworzenia gry, przykładową stroną, która udostępnia tego typu grafikę może być OpenGameArt [41]. Jest to popularna strona umożliwiająca publikowanie własnych treści takich jak zestawy kafelków. Na rysunku 9.1 pokazana została przykładowa lista zasobów dostępnych na stronie.



Rys 9.1. Screenshot ze strony OpenGameArt

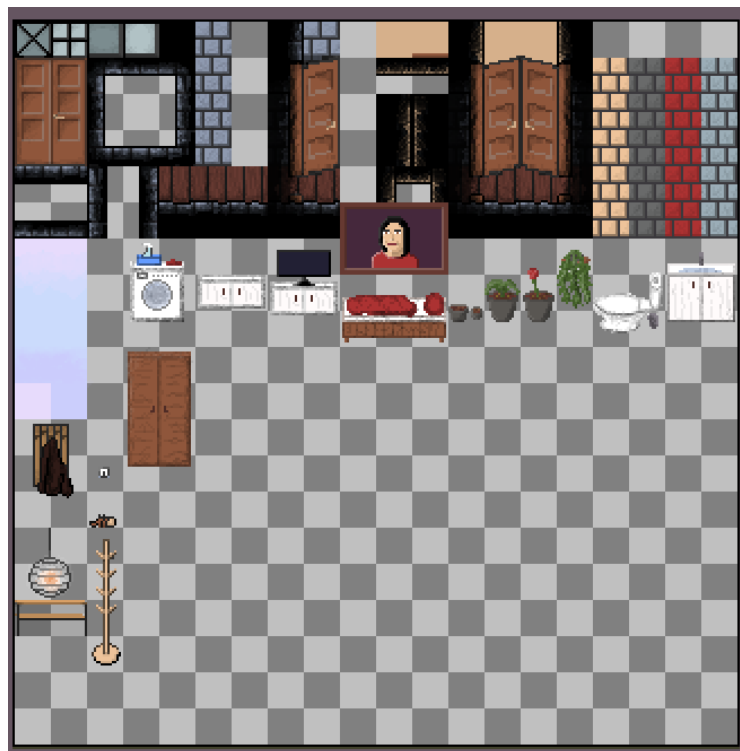
Oprócz pożytkowania się darmowymi treściami można również przygotować własne przy pomocy rozmaitych programów takich jak Krita [42], Aseprite [40] (rys. 9.2). Są to zaawansowane oprogramowania oferujące duże możliwości obróbki obrazów czy nawet wykonywania własnych animacji.



Rys 9.2. Screenshot z programu Aseprite

9.2. Oprawa graficzna map

Popularnym rozwiązaniem przy budowie map w grach dwuwymiarowych jest wykorzystywanie zestawów kafelków, czy to stworzonych własnoręcznie, pobranych z darmowych źródeł, przykładowy został pokazany na rysunku numer 9.3.



Rys 9.3. Zrzut ekranu przedstawiający zestaw kafelków w Aseprite

9.2.1. Mapa Dom

Mapa ta składa się z kilku scen pomiędzy którymi gracz może przechodzić za pomocą drzwi. Przedstawione sceny to między innymi sypialnia (rys. 9.4), korytarz (rys. 9.5), łazienka (rys. 9.6) i pokój wejściowy (rys. 9.7).



Rys 9.4. Zrzut ekranu przedstawiający scenę sypialni



Rys 9.5. Zrzut ekranu przedstawiający scenę głównego korytarza



Rys 9.6. Zrzut ekranu przedstawiający scenę ubikacji



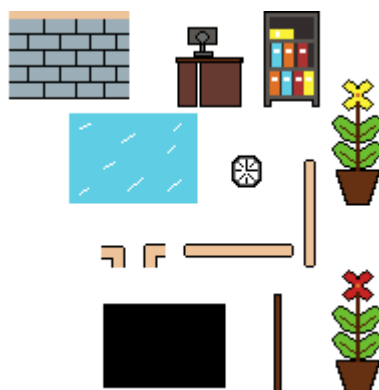
Rys 9.7. Zrzut ekranu przedstawiający pokój wyjściowy

9.2.2. Mapa Biura

Mapa przedstawiająca biuro, w którym pracuje protagonista całej historii oraz zestaw kafelek użytych do zbudowania jej (rys. 9.8 i 9.9).



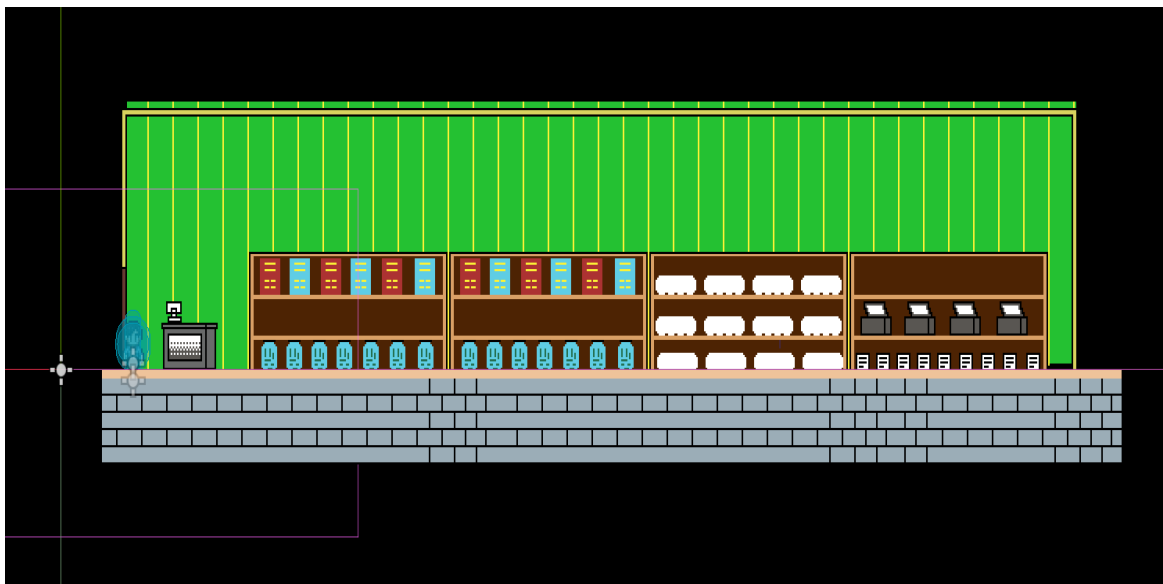
Rys 9.8. Zrzut ekranu przedstawiający pokój biura



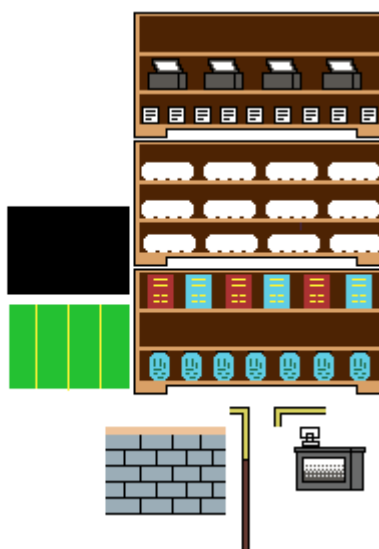
Rys 9.9. Zrzut ekranu przedstawiający własnoręcznie wykonane zasoby w formie pliku zbioru kafelków

9.2.3. Mapa supermarketu

Poniżej na rysunku 9.10. jest przedstawiona mapa supermarketu, w którym odbywa się część fabularna gry oraz walka z pierwszym większym przeciwnikiem. Rysunek 9.11. pokazuje zbiór kafelek użyty do zbudowania owej mapy.



Rys. 9.10. Scena supermarketu.



Rys. 9.11. Zestaw kafelek o nazwie Supermarket.png

9.2.4. Mapa nocy

Mapa ukazująca się pod koniec dnia i mająca być podsumowaniem tego czego gracz osiągnął. Ukazana jest na rysunku 9.12, a poniżej na rysunku 9.13, 9.14 i 9.15 jest zbiór kafelek użytych do zbudowania sceny.



Rys. 9.12. Scena nocy.



Rys. 9.13. Zestaw kafelków użytych w scenie nocy.



Rys. 9.14. Zestaw kafelek użytych w scenie nocy.



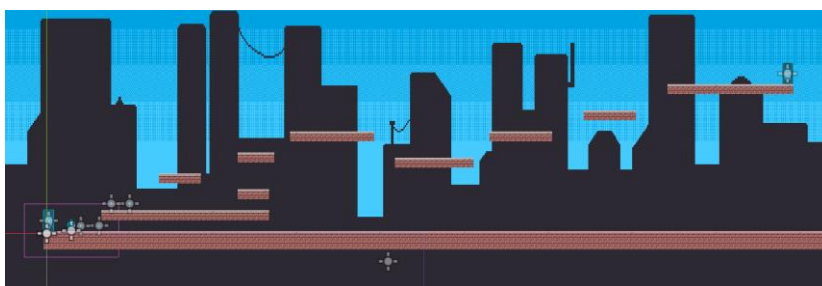
Rys. 9.15. Tło użyte w scenie nocy.

9.2.5. Mapy platformowe.

Mapy, które trzeba pokonać by przejść do następnego etapu rozgrywki. Zawarci są w nich przeciwnicy, z którymi gracz może stoczyć walkę. Przykładowe mapy są podane w rysunkach 9.16 i 9.17.



Rys. 9.16. Pierwsza scena platformowa.



Rys. 9.17. Druga scena platformowa.

9.3. Postacie nie grywalne i przeciwnicy.

9.3.1. Postacie nie grywalne.

Postacie istniejące w grze, by wypełnić przestrzeń map, w których można skorzystać z dialogu. Poniżej znajdują się zbiory przykładowych rysunków 9.18, 9.19, 9.20 i 9.21 wykorzystywanych do animacji tych postaci.



Rys.9.18. Zestaw rysunków do animacji postaci numer 1.



Rys. 9.19. Zestaw rysunków do animacji postaci numer 2.



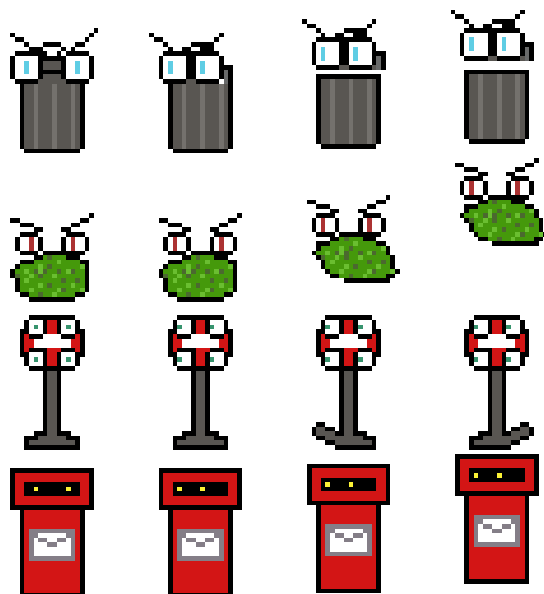
Rys. 9.20. Zestaw rysunków do animacji postaci numer 3.



Rys. 9.21. Zestaw rysunków do animacji postaci numer 4.

9.3.2. Przeciwnicy.

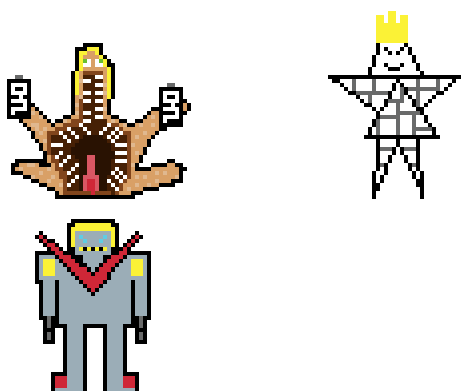
Są to postacie, które podczas walki będą próbowali przeszkodzić graczowi w osiągnięciu celu. Rysunek 9.22 przedstawia czterech przeciwników obecnych podczas gry na poziomach platformowych i w scenie walki.



Rys. 9.22. Rysunek postaci przeciwników użytych w grze.

9.3.3. Więksi przeciwnicy.

Pod koniec dnia w grze gracz musi stoczyć walkę z większym przeciwnikiem, by dostać się do końca. Poniżej jest rysunek 9.23, który przedstawia owych wrogów.

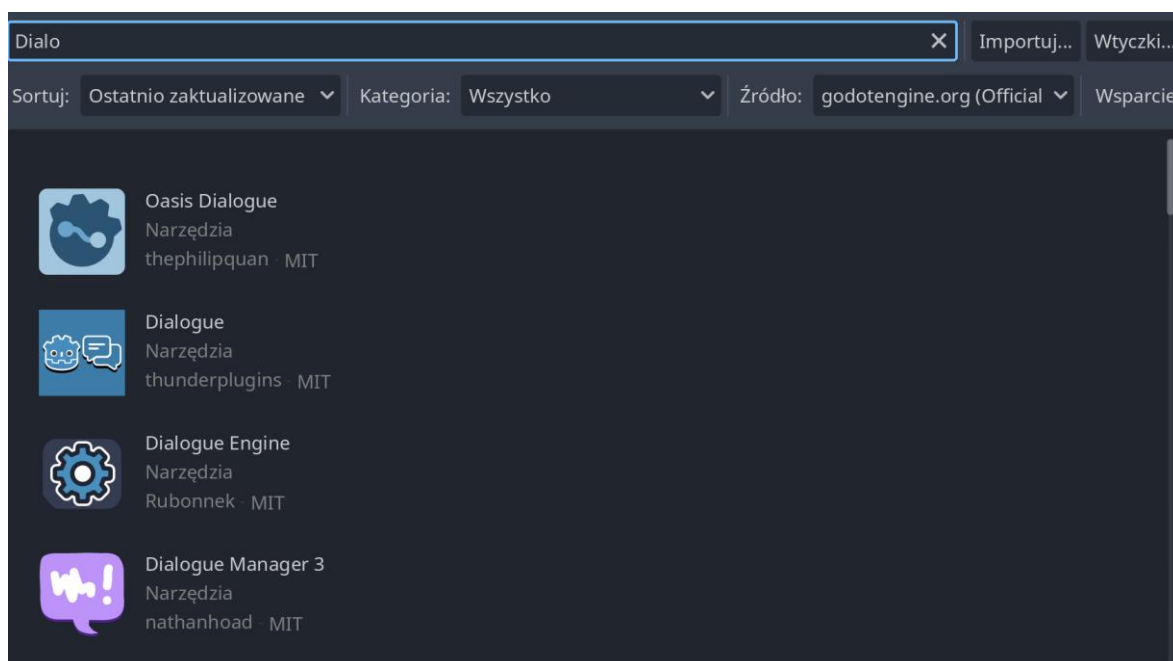


Rys. 9.23. Rysunek większych przeciwników użytych w grze.

10. Implementacja głównych mechanik

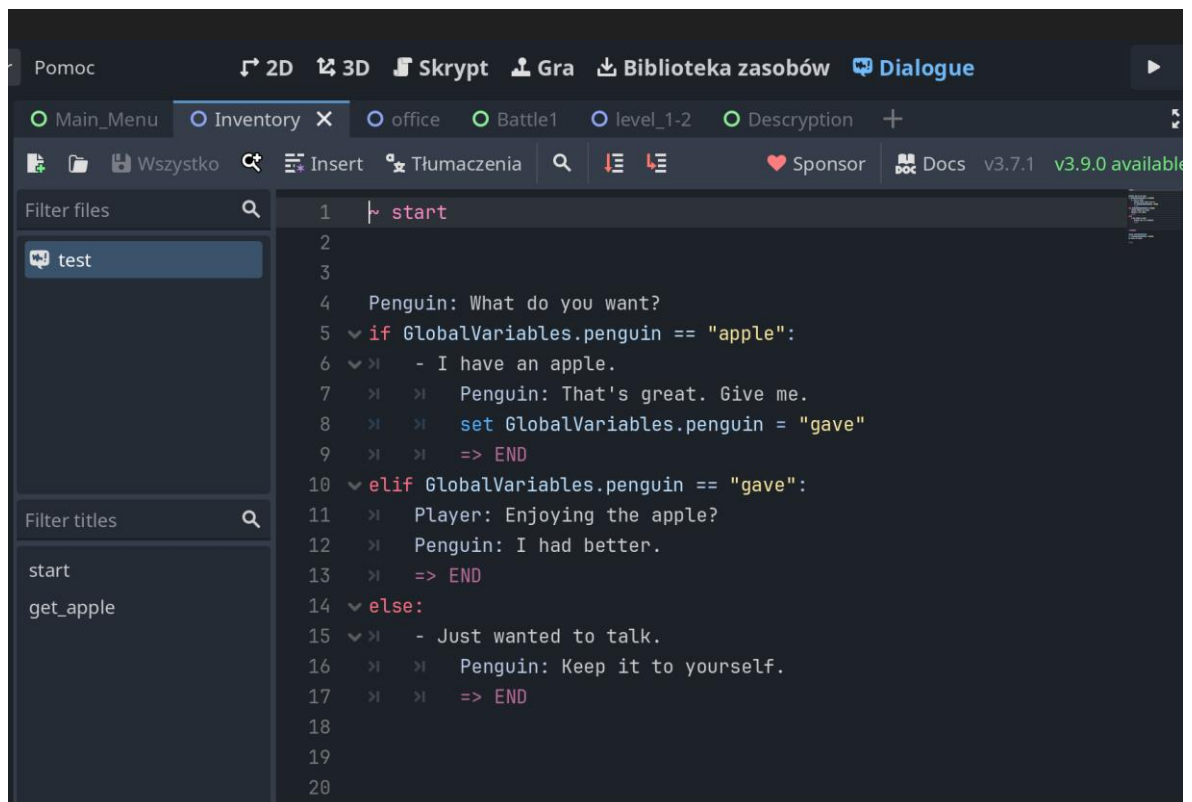
10.1. Menadżer dialogu

Jeden z ważniejszych elementów gry stosowany do stworzenia interakcji pomiędzy postacią gracza, a nie grywalną postacią. Wykorzystywany również do wprowadzenia gracza do zadań pobocznych, które mogą, ale nie muszą być wykonane, by dostać inne zakończenie. Cały menedżer został zaimplementowany do Godot'a [6] poprzez Bibliotekę Zasobów z pakietu Dialogue Manager 3 przedstawionego na rysunku 10.1.



Rys. 10.1. Pakiet Dialogue Manager 3 w Bibliotece Zasobów.

Cała interakcja jest zapisana w specjalnym pliku `.dialogue` (rys. 10.2), z którego owa interakcja pochodzi, a dialog można uruchomić wchodząc w kolizję z postacią niegrywalną i naciskając klawisz E na klawiaturze, co spowoduje rozpoczęcie działania skryptu `actionable.gd` zawartego w listingu 10.1, który rozpoczyna dialog.



Rys. 10.2 . Przykład pliku .dialogue

`extends Area2D`

```
const Balloon = preload("res://dialogue/balloon.tscn")
@export var dialogue_resource: DialogueResource
@export var dialogue_start: String = "start"

func action() -> void:
    #print("ballon started", !GlobalVariables.has_dialog_started)
    if !GlobalVariables.has_dialog_started:
        DialogueManager.show_example_dialogue_balloon(dialogue_resource,
dialogue_start)
    #var bal: Node = Balloon.instantiate()
    #bal.start(dialogue_resource, dialogue_start)
```

Listing 10.1. Kod pliku actionable.gd

10.2. Mechanika zmiany scen w trakcie rozgrywki

W grze została zaimplementowana funkcjonalność pozwalająca dla gracza na sprawne przechodzenie między scenami za pomocą specjalnych obiektów nazywanych drzwiami. Cały proces jest zarządzany za pomocą Menedżera Drzwi, współpracującego z kolei z Menedżerem Scen. Cały system w znaczącym stopniu polega na sygnałach, będących integralną częścią całego silnika Godot Engine. Służą one do komunikacji między obiektami różnych klas, reprezentujących poszczególne węzły scen gry. Takie podejście pozwala na ograniczenie zależności między węzłami i scenami, które nie są widoczne między sobą lub nie mają bezpośredniego dostępu do siebie.

Scena mapy gry jest zawsze osadzona w węźle “Levels”, będącym częścią głównej sceny “Gameplay”. Natomiast wewnątrz “Levels” znajduje się z kolei instancja postaci gracza, która może poruszać się po mapie wraz z siedzącą ją kamerą. W przypadku wejścia przez nią w obręb instancji dowolnych drzwi zostaje zainicjowany cały proces przechodzenia przez drzwi, a w przypadku wyjścia z nich zostaje on zakończony. Co dzieje się można zobrazować poprzez przesiedzenie kodu odpowiedzialnego za działanie tego systemu.

```
enum Levels_Enum {
    one, two,

    house_entry, house_level, house_toilet, house_bedroom,

    path_one_two, office
}

const LEVELS = {
    Levels_Enum.one: "res://scenes/Maps/level_one.tscn",
    Levels_Enum.two: "res://scenes/Maps/level_two.tscn",

    Levels_Enum.house_entry: "res://scenes/Maps/house/house_entry.tscn",
    Levels_Enum.house_level: "res://scenes/Maps/house/house_level.tscn",
    Levels_Enum.house_toilet: "res://scenes/Maps/house/house_toilet.tscn",
    Levels_Enum.house_bedroom: "res://scenes/Maps/house/house_bedroom.tscn",

    Levels_Enum.path_one_two: "res://scenes/Maps/level_1-2.tscn",
    Levels_Enum.office: "res://scenes/Maps/office.tscn",
}
```

Listing 10.2. Definicja enum i const “LEVELS” w doors.gd

```

enum Transition_Enum {
    fade_from_black,
    fade_from_white,
    no_to_transition,
}

const TRANSITION_VALUES = {
    Transition_Enum.fade_from_black: "fade_from_black",
    Transition_Enum.fade_from_white: "fade_from_white",
    Transition_Enum.no_to_transition: "no_to_transition",
}

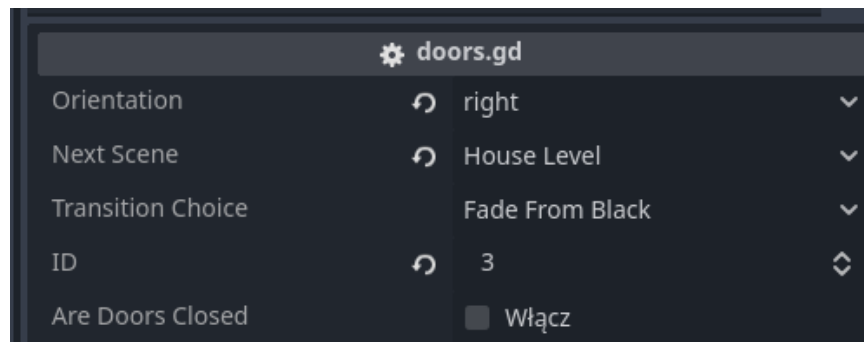
@onready var collision_area = $Area2D
@onready var collision_shape = $Area2D/CollisionShape2D

@export_enum("left", "right", "vertical") var orientation
@export var next_scene: Levels_Enum
@export var transition_choice: Transition_Enum
@export_range(0, 10) var id : int
@export var are_doors_closed: bool = false

```

Listing 10.3. Pozostałe definicje i deklaracje zmiennych, stałych i wyliczeń w doors.gd

Najważniejszą częścią listingów 10.3 i 10.2 są zmienne z adnotacją “export”, oznaczającą w praktyce możliwość ustawienia wartości tej zmiennej w interfejsie graficznym silnika gry dla każdej instancji drzwi osobno, co pozwala na określenie działania każdego z drzwi z osobna. Jest to przedstawione na rysunku 10.3 i jak widać w kolejności od góry do dołu, możliwe jest wybranie orientacji drzwi na prawo, lewo lub horyzontalnie. Oprócz tego umożliwione jest wybranie następnej sceny, czyli docelowej do której postać gracza zostanie przetransportowana. Kolejno umożliwione zostało wybranie animacji tranzycji między scenami, służącej do wygaszania ekranu i identyfikator drzwi, zarówno tych do których postać zostanie przetransportowana i tych do których przypisana jest ta zmienna.



Rys. 10.3. Zmienne doors.gd w interfejsie silnika Godot Engine.

```

func _ready() -> void:
    DoorManager.all_door_registrate_in_scene_manager.emit(self, id)

func _on_area_2d_body_entered(body: Node2D) -> void:
    var body_Name = body.name

    if (body_Name != "Player"):
        collision_area.monitorable = false
    elif DoorManager.did_player_go_through_doors:
        pass
    elif are_doors_closed:
        pass
    elif (orientation != null && next_scene != null && transition_choice
          != null && id != null
        ):
        DoorManager.player_entered_doors.emit(id, orientation,
        TRANSITION_VALUES[transition_choice],
        LEVELS[next_scene], position
        )

    else:
        printerr("nie uzupełnione wartosci drzwi: ", self.name)

func _on_area_2d_body_exited(body: Node2D) -> void:
    if !DoorManager.did_player_go_through_doors:
        DoorManager.player_exited_doors.emit()

```

Listing 10.4. Metody w doors.gd

Funkcja “_on_area_2d_body_entered” z listingu 10.4 zostaje wywoływana za każdym razem, kiedy dowolne ciało wejdzie w kolizję z kształtem węzła, do którego jest przypisana. Dalej zostaje sprawdzone czy ciało te należy do postaci gracza i jeśli jest to prawdą i wszystkie inne warunki są spełnione, w tym w szczególności, że zmienne z adnotacją export mają wartość, to wyemitowany zostaje sygnał do Menedżera Drzwi przekazujący te zmienne. Dalszy proces przenoszenia gracza między scenami map jest wykonywany wewnątrz tego singletona. Oprócz tego warto zwrócić uwagę na analogiczną funkcję “_on_area_2d_body_exited” emitującą sygnał do tego samego menedżera informujący o wyjściu przez postać gracza z danych drzwi.

```

extends Node

signal player_entered_doors(door_id: int, door_orientation: int,
    transition_name: String, next_map_name: String,
    door_position: Vector2)
signal player_exited_doors()
signal player_ready_to_pass(timer_time: float)
signal all_door_registrate_in_scene_manager(door_node: Node2D,
    door_id: int)
signal player_ready_to_teleport_to_door()
signal player_ready()

var level_Container: Node2D = null

var is_player_inside_vertical_doors = false
var is_player_inside_horizontal_doors = false
var did_player_go_through_doors = false

var door_dictionary = {
    "door_id": 0,
    "door_orientation": 0,
    "transition_name": "",
    "next_map_name": "",
    "door_position": Vector2(0, 0)
}

#door_list contains all instances of doors that emitted the signal called
#all_door_registrate_in_scene_manager
var door_list = [null, null, null, null, null, null, null, null, null, null,
    ,null]

func _ready() -> void:
    player_entered_doors.connect(self._on_player_in_doors)
    player_exited_doors.connect(self._on_player_exited_doors)
    player_ready_to_pass.connect(self._on_player_ready_to_pass)
    all_door_registrate_in_scene_manager.connect(
        self._on_all_door_registrate_in_scene_manager)
    player_ready.connect(self._on_player_ready)

    _on_player_exited_doors()

func _on_player_in_doors(door_id: int, door_orientation: int,
    transition_name: String, next_map_name: String,
    door_position: Vector2):
    if !(is_player_inside_vertical_doors ||
        is_player_inside_horizontal_doors) :
        if door_orientation == 2:

```



```

        is_player_inside_vertical_doors = true
    elif (door_orientation == 0 || door_orientation == 1):
        is_player_inside_horizontal_doors = true
    else:
        _on_player_exited_doors()
#these doors are not configured properly
    return

    door_dictionary = {
        "door_id": door_id,
        "door_orientation": door_orientation,
        "transition_name": transition_name,
        "next_map_name": next_map_name,
        "door_position": door_position
    }
else:
    pass

func _on_player_exited_doors():
    is_player_inside_vertical_doors = false
    is_player_inside_horizontal_doors = false
    did_player_go_through_doors = false

func _on_all_door_registrate_in_scene_manager(door_node: Node2D, door_id:
int):
    if door_list[door_id] == null:
        door_list[door_id] = door_node
    else:
        pass

func _clear_door_list():
    door_list = [null, null, null, null, null, null, null, null, null
,null]

func _on_player_ready_to_pass(timer_time: float):
    await get_tree().create_timer(timer_time).timeout
    _clear_door_list()
    print(door_dictionary["next_map_name"],level_Container,
        level_Container.get_child(0),door_dictionary["transition_name"]
    )
    did_player_go_through_doors = true
    is_player_inside_vertical_doors = false
    is_player_inside_horizontal_doors = false
    SceneManager.swap_scenes(door_dictionary["next_map_name"],
        level_Container,level_Container.get_child(0),
        door_dictionary["transition_name"]
    )

```

```

func _on_player_ready():
    if did_player_go_through_doors:
        var door_target = door_list[door_dictionary["door_id"]]
        if door_target == null:
            did_player_go_through_doors = false
            _on_player_exited_doors()
        else:
            if door_target.orientation != null:
                door_dictionary = {
                    "door_id": door_target.id ,
                    "door_orientation": door_target.orientation,
                    "transition_name": "",
                    "next_map_name": "",
                    "door_position": door_target.position
                }
                player_ready_to_teleport_to_door.emit()
            else:
                did_player_go_through_doors = false
                _on_player_exited_doors()

```

Listing 10.5. Kod źródłowy pliku DoorManager.gd

Na obecnym etapie w listingu o numerze 10.5 najważniejsze są dwie funkcje “_on_player_in_doors” i “_on_player_exited_doors”, ponieważ są one odpowiednio wywoływane przy otrzymaniu wcześniej otrzymanych sygnałów. Ustawiają one lub zerują flagi używane przez Menedżera Drzwi i maszynę stanów gracza, o której można przeczytać więcej w rozdziale 10.5 o niej opowiadającym. Oprócz tego przy wejściu do drzwi przez gracza menedżer zapisuje dane aktualnie używanych drzwi w słowniku “door_dictionary”.

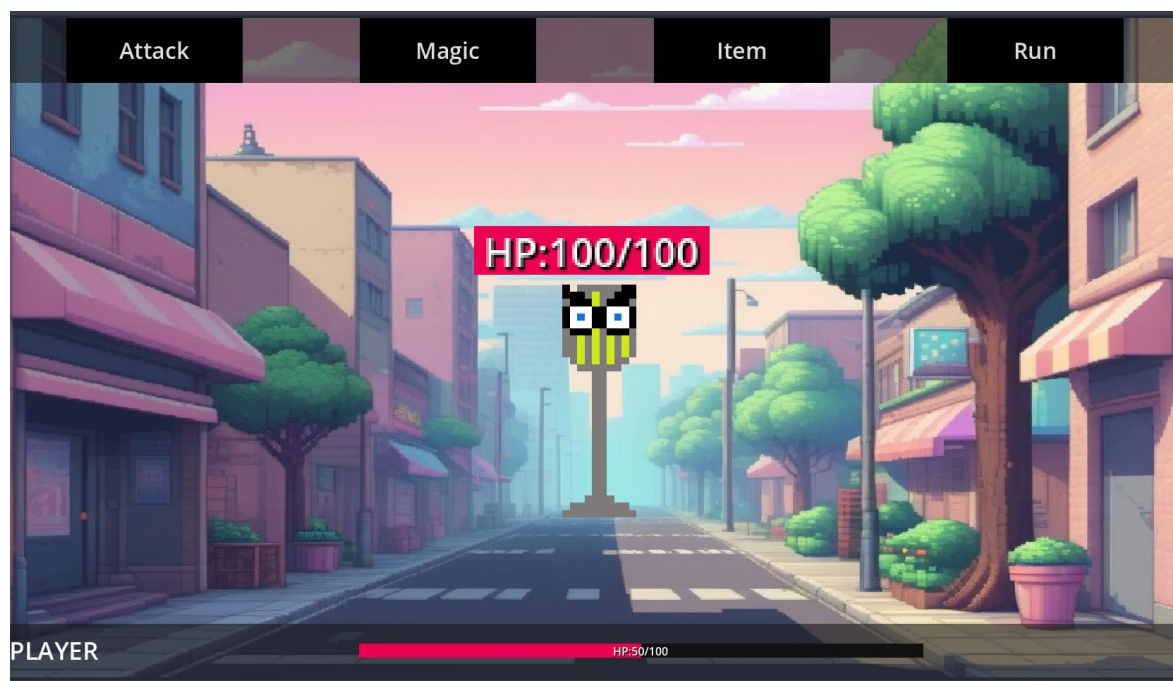
Z kolei maszyna stanów postaci gracza, po ustawieniu jednej z flag “is_player_inside_vertical_doors” lub “is_player_inside_horizontal_doors” przez menedżera, rozpoczyna specjalny stan wchodzenia do drzwi, w obrębie którego sprawdzane będzie położenie postaci i z każdą klatką będzie ona kierować się w stronę środka drzwi. Ważne jest podkreślenie, że przy ustawionej fladze “is_player_inside_horizontal_doors” gracz musi dodatkowo nacisnąć przycisk gry “Interact”. Po spełnieniu odpowiednich wymogów zostaje wysłany sygnał od gracza o nazwie “player_ready_to_pass” połączony z funkcją “_on_player_ready_to_pass” z listingu 10.5. W tej funkcji wywoływana jest metoda Menedżera Scen, który z kolei przeprowadza tranzycję między scenami map i je podmienia.

Postać gracza po zmianie sceny zostaje teleportowana do nowych drzwi i rozpoczyna się jej stan wychodzenia z drzwi. W trakcie tego procesu flagi Menegera Drzwi

są zerowane i postać może być sterowana przez gracza znowu w normalny sposób.

10.3. System walki turowej

Poza prostymi poziomami platformowymi gra oferuje również interakcje z nie grywalnymi postaciami w tych poziomach, by rozpocząć walkę z nimi. Przy wejściu w kolizję z przeciwnikiem gracz zostaje przeniesiony do sceny mającej reprezentować walkę z przeciwnikiem (rys. 10.4). W centrum samej sceny jest postać, z którą walczysz oraz jej pasek życia, poniżej znajduje się nazwa gracza oraz pasek reprezentujący zdrowie gracza. W górnej części sceny znajdują się cztery przyciski reprezentujące dowolne akcje jakie gracz może wykonać w tej sytuacji. Pierwszy przycisk z lewej strona rozpocznie atak gracza, który jest wartością mnożącą podstawową siłę ataku gracza z mocą broni jaką posiada, co spowoduje zmianę punktów zdrowia u przeciwnika. Drugi w kolejności przycisk reprezentuje dostępne zaklęcia jakie gracz może rzucać na przeciwnika, niektóre z nich nakładają na niego specjalne efekty, które z czasem zadają przeciwnikowi masę obrażeń. Trzeci przycisk jest wglądem na ekwipunek gracza, który oferuje wszystkie dotąd zebrane przedmioty, których użycie może wyleczyć gracza, jak i zranić przeciwnika. Ostatnim przyciskiem jest ucieczka, która pozwala graczowi wyjść ze sceny walki i powrócić do poziomu platformowego. Gracz podczas walki, może wykonać tylko jedną akcję, po której przeciwnik zaatakuje protagonistę i cała pętla będzie kontynuowana, aż do momentu, w którym jedna z postaci polegnie.



Rys. 10.4 Scena walki turowej

Poniżej przedstawione są poszczególne funkcje w listingu 10.6 mającym reprezentować skrypt battle.gd. Funkcja `_ready` wykonuje kod jeszcze przed uruchomieniem sceny, co wraz z sygnałem do funkcji `_player_joins_battle` wstawia obecne zmienne gracza i przeciwnika. Funkcja `input` ukrywa kontener tekstowy za każdym razem, kiedy lewy przycisk myszy jest wciśnięty. Funkcja `set_health` ustawia nową wartość dla paska postępowego mającego reprezentować zdrowie przeciwnika lub bohatera. Funkcja `show_text` uzupełnia `input` pokazując tekst w kontenerze, a `enemy_turn` reprezentuje moment, kiedy akcja została wykonana przez gracza i przeciwnik go atakuje. `On_run_pressed` obsługuje przycisk powodując powrót do sceny poziomego platformowego, natomiast `on_attack_pressed` wykonuje obrażenia na przeciwniku, za każdym razem, gdy jest naciśnięty przycisk ataku. `On_magic_pressed` pokazuje panel z odblokowanymi zaklęciami, które są obsługiwane przez poszczególne funkcje. `Fireball_pressed` tworzy iluzję potężnego ataku zadającego obrażenia wrogowi, a `poison_pressed` wraz z funkcją `poison` zadaje małe obrażenia natomiast wywołuje efekt, który co dwie sekundy zadaje 5 punktów obrażeń. Funkcja `bleed_pressed` wywołuje ten sam efekt co `poison_pressed`, tylko z funkcją `bleed` wywołuje 20 punktów obrażeń co 30 sekund. `_On_item_pressed` tworzy instancje sceny `button` za każdym razem, kiedy znajdzie przedmiot w ekwipunku i dodaje je do kontenera, natomiast funkcja `hide_item` ukrywa panel zawierający ten kontener. `Dead` we współpracy z `_end_battle` tworzy obsługuje ucieczkę gracza oraz moment, w których punkty życia gracza lub przeciwnika są równe zero.

`extends Control`

```
signal item_press
signal text_closed
signal death
@export var enemy: Resource = null
var player_health = GlobalVariables.curr_health
@export var e_health: int
@export var e_exp: int
const InventoryClass = preload("res://scripts/Inventory/PlayerInventory.gd")
const ItemClass = preload("res://scripts/Inventory/Item_script.tres.gd")
@onready var button = preload("res://scenes/Battle/Button1.tscn")

func _ready() -> void:
    SignalBus.is_in_battle.connect(_player_joins_battle)

    visible = false
```

```

death.connect(dead)
SignalBus.hiding.connect(hide_item)
SignalBus.health_up.connect(healing_item)
item_press.connect(_on_item_pressed)

func _input(event) -> void:
    if (Input.is_action_just_pressed("Interact") or
Input.is_mouse_button_pressed(MOUSE_BUTTON_LEFT)) and $Textbox.visible:
        $Textbox.hide()
        emit_signal("text_closed")

func set_health(progress_bar, health, max_health) -> void:
    progress_bar.value = health
    progress_bar.max_value = max_health
    progress_bar.get_node("Label").text = "HP:%d/%d" % [health, max_health]

func healing_item():
    set_health($PlayerPanel/HBoxContainer/ProgressBar,
GlobalVariables.curr_health, GlobalVariables.health)

func show_text(text):
    $Textbox.show()
    $Textbox/Label.text = text

func enemy_turn() -> void:
    show_text("%s charges at you" % enemy.name)
    await text_closed
    GlobalVariables.curr_health = max(0, GlobalVariables.curr_health -
(enemy.damage-
(GlobalVariables.player_shirt*GlobalVariables.player_pants*GlobalVariables.p
layer_boots)))
    set_health($PlayerPanel/HBoxContainer/ProgressBar,
GlobalVariables.curr_health, GlobalVariables.health)
    $AnimationPlayer.play("player_damage")
    await $AnimationPlayer.animation_finished
    dead()

func _on_run_pressed() -> void:
    $ActionPanel/HBoxContainer/Magic/MagicPanel.hide()
    $ActionPanel.hide()
    show_text("You ran away")
    _end_battle(false)

func _on_attack_pressed() -> void:
    show_text("You attack")

```

```

        await text_closed
        e_health = max(0, e_health -
((GlobalVariables.sword*GlobalVariables.attack_power)))
        set_health($EnemyContainer/ProgressBar, e_health, enemy.health)
        $AnimationPlayer.play("enemy_damage")
        await $AnimationPlayer.animation_finished
        show_text("You dealt %d damage." %
(GlobalVariables.sword*GlobalVariables.attack_power))
        await text_closed
        if $EnemyContainer/ProgressBar.value == 0:
            dead()
        else :
            enemy_turn()

func _on_magic_pressed() -> void:
    $ActionPanel/HBoxContainer/Magic/MagicPanel.show()
    if GlobalVariables.fireball:

$ActionPanel/HBoxContainer/Magic/MagicPanel/MagicContainer/Button/MagicButto
n.text = GlobalVariables.f_name

$ActionPanel/HBoxContainer/Magic/MagicPanel/MagicContainer/Button/MagicButto
n/Label.text = ""

$ActionPanel/HBoxContainer/Magic/MagicPanel/MagicContainer/Button.show()
    var b =
$ActionPanel/HBoxContainer/Magic/MagicPanel/MagicContainer/Button/MagicButto
n
        b.connect("pressed", fireball_pressed)
    if GlobalVariables.poison_sting:

$ActionPanel/HBoxContainer/Magic/MagicPanel/MagicContainer/Button2/MagicButt
on.text = GlobalVariables.p_name

$ActionPanel/HBoxContainer/Magic/MagicPanel/MagicContainer/Button2/MagicButt
on/Label.text = ""

$ActionPanel/HBoxContainer/Magic/MagicPanel/MagicContainer/Button2.show()
    var c =
$ActionPanel/HBoxContainer/Magic/MagicPanel/MagicContainer/Button2/MagicButt
on
        c.connect("pressed", poison_pressed)
    if GlobalVariables.sting_ray:

$ActionPanel/HBoxContainer/Magic/MagicPanel/MagicContainer/Button3/MagicButt
on.text = GlobalVariables.b_name

```

```

$ActionPanel/HBoxContainer/Magic/MagicPanel/MagicContainer/Button3/MagicButt
on/Label.text = ""

$ActionPanel/HBoxContainer/Magic/MagicPanel/MagicContainer/Button3.show()
    var d
=$ActionPanel/HBoxContainer/Magic/MagicPanel/MagicContainer/Button3/MagicBut
ton
    d.connect("pressed", bleed_pressed)

func fireball_pressed():
    $ActionPanel/HBoxContainer/Magic/MagicPanel.hide()
    show_text("You use fireball")
    await text_closed
    e_health = max(0, e_health - GlobalVariables.f_damage)
    set_health($EnemyContainer/ProgressBar, e_health, enemy.health)
    $AnimationPlayer.play("enemy_damage")
    await $AnimationPlayer.animation_finished
    show_text("You dealt %d damage." % GlobalVariables.f_damage)
    await text_closed
    if $EnemyContainer/ProgressBar.value == 0:
        dead()
    else :
        enemy_turn()

func poison_pressed():
    $ActionPanel/HBoxContainer/Magic/MagicPanel.hide()
    show_text("You use poison sting")
    await text_closed
    e_health = max(0, e_health - GlobalVariables.p_damage)
    set_health($EnemyContainer/ProgressBar, e_health, enemy.health)
    $AnimationPlayer.play("enemy_damage")
    await $AnimationPlayer.animation_finished
    show_text("You dealt %d damage." % GlobalVariables.p_damage)
    await text_closed
    _posion()
    if $EnemyContainer/ProgressBar.value == 0:
        dead()
    else :
        enemy_turn()

func bleed_pressed():
    $ActionPanel/HBoxContainer/Magic/MagicPanel.hide()
    show_text("You use sting ray")
    await text_closed
    e_health = max(0, e_health - GlobalVariables.b_damage)
    set_health($EnemyContainer/ProgressBar, e_health, enemy.health)
    $AnimationPlayer.play("enemy_damage")

```

```

await $AnimationPlayer.animation_finished
show_text("You dealt %d damage." % GlobalVariables.b_damage)
await text_closed
_bleed()
if $EnemyContainer/ProgressBar.value == 0:
    dead()
else :
    enemy_turn()

func _posion():
    if get_tree().current_scene.name == "Battle1":
        for i in range(15):
            await get_tree().create_timer(2.0).timeout
            e_health = max(0, e_health - GlobalVariables.p_effect)
            set_health($EnemyContainer/ProgressBar, e_health, enemy.health)
            if $EnemyContainer/ProgressBar.value == 0:
                dead()
                break

func _bleed():
    if get_tree().current_scene.name == "Battle1":
        for i in range(2):
            await get_tree().create_timer(30.0).timeout
            e_health = max(0, e_health - GlobalVariables.b_effect)
            set_health($EnemyContainer/ProgressBar, e_health, enemy.health)
            if $EnemyContainer/ProgressBar.value == 0:
                dead()
                break

func _on_item_pressed():
    var inv = PlayerInventory.inventory
    var y = 0
    if
$ActionPanel/HBoxContainer/Item/ItemPanel/ItemContainer.get_child_count() ==
0:
        for i in inv:
            y += 50
            var button_instance = button.instantiate()
            var button_name = button_instance.get_node("MagicButton")
            button_name.text = inv[i][0]
            var button_ammount =
button_instance.get_node("MagicButton/Label")
            button_ammount.text = str(inv[i][1])
            #button_instance.position = Vector2(250, y)
            print("Hello1")
            $ActionPanel/HBoxContainer/Item/ItemPanel.size = Vector2(250, y)

$ActionPanel/HBoxContainer/Item/ItemPanel/ItemContainer.add_child(button_ins

```



```

tance)

$ActionPanel/HBoxContainer/Item/ItemPanel/ItemContainer.add_spacer(false)
    print("Hello2")
else :
    $ActionPanel/HBoxContainer/Item/ItemPanel.show()
    var items =
$ActionPanel/HBoxContainer/Item/ItemPanel/ItemContainer.get_children()
    for i in range(0, items.size(), 2):
        if items[i] == button:
            var label_count = items[i].get_node("MagicButton/Label")
            if label_count.text == "0":
                items.remove_child(items[i])

func hide_item():
    $ActionPanel/HBoxContainer/Item/ItemPanel.hide()

func dead():
    if $EnemyContainer/ProgressBar.value == 0:
        $AnimationPlayer.play("enemy_dead")
        await $AnimationPlayer.animation_finished
        GlobalVariables.health =
$PlayerPanel/HBoxContainer/ProgressBar.value
        show_text("The %s has been defeated." % enemy.name)
        await text_closed
        PlayerLevel.exp_get(e_exp)
        if PlayerLevel.current_exp >= PlayerLevel.level_2 and
PlayerLevel.player_level == 1:
            show_text("You are now level 2.")
            await text_closed
            show_text("You have learnt Poison Sting")
            await text_closed
            PlayerLevel.new_level.emit()
            elif PlayerLevel.current_exp >= PlayerLevel.level_3 and
PlayerLevel.player_level == 2:
                show_text("You are now level 3.")
                await text_closed
                show_text("You have learnt Sting Ray")
                await text_closed
                PlayerLevel.new_level.emit()
            await get_tree().create_timer(0.25).timeout
            #get_tree().quit()
            _end_battle(false)
        elif $PlayerPanel/HBoxContainer/ProgressBar.value == 0:
            show_text("The Player has been defeated.")
            await text_closed
            await get_tree().create_timer(0.25).timeout
            #get_tree().quit()

```

```

        _end_battle(true)

#szymi
##jest używane przy ucieczce, śmierci gracza lub przeciwnika
func _end_battle(player_died: bool):
    visible = false
    GlobalVariables.is_in_battle = false
    SignalBus.battle_ended.emit()
    if player_died:
        SceneManager.swap_scenes("res://scenes/UI/Main_Menu.tscn"
, get_tree().root , self.get_parent().get_parent() ,"no_to_transition")
        #get_tree().quit()

func _player_joins_battle():
    # $ActionPanel/HBoxContainer/Item.emit(item_press)
    set_health($EnemyContainer/ProgressBar, enemy.health, enemy.health)
    set_health($PlayerPanel/HBoxContainer/ProgressBar,
GlobalVariables.curr_health, GlobalVariables.health)
    $EnemyContainer/Enemy.texture = enemy.texture
    e_health = enemy.health
    e_exp = enemy.enemy_exp
    $Textbox.hide()
    $ActionPanel.hide()
    $ActionPanel/HBoxContainer/Magic/MagicPanel.hide()
    $ActionPanel/HBoxContainer/Magic/MagicPanel/MagicContainer/Button.hide()

$ActionPanel/HBoxContainer/Magic/MagicPanel/MagicContainer/Button2.hide()

$ActionPanel/HBoxContainer/Magic/MagicPanel/MagicContainer/Button3.hide()
    for i in PlayerInventory.equips:
        if PlayerInventory.equips[i][3] == "Shirt":
            GlobalVariables.player_shirt = PlayerInventory.equips[i][4]
        if PlayerInventory.equips[i][3] == "Pants":
            GlobalVariables.player_pants = PlayerInventory.equips[i][4]
        if PlayerInventory.equips[i][3] == "Shoes":
            GlobalVariables.player_boots = PlayerInventory.equips[i][4]
        if PlayerInventory.equips[i][3] == "Sword":
            GlobalVariables.sword = PlayerInventory.equips[i][4]
    visible = true
    $AnimationPlayer.play("RESET")
    show_text("A wild %s appears" % enemy.name)
    await text_closed
    show_text("Stop right there")
    await text_closed
    $ActionPanel.show()

```

10.4. System ekwipunku

Wszystkie przedmioty, które zostały wcześniej wspomniane w podrozdziale o systemie walki turowej są obsługiwane przez system ekwipunku pokazanego na rysunku 10.5. Cały system składa się z dwóch scen i czterech skryptów odpowiedzialnych za poszczególne elementy. Scena Inventory pokazuje kontener zawierając dwadzieścia cztery panele odpowiedzialne za przechowanie określonej ilości przedmiotów w ekwipunku, natomiast po lewej stronie są przedmioty, które gracz “zakłada” by zdobyć specjalne bonusy.



Rys. 10.5 Scena ekwipunku

Poniżej jest przedstawiony skrypt odpowiedzialny za przedmiot o nazwie Item_tres.gd zwarty w listingu 10.7. Funkcja `_ready` tworzy przedmiot na podstawie obrazu z katalogu Items i danych zawartych w ItemData.json, natomiast funkcja `set_item` przydziela przedmiot do rubryki w ekwipunku. Pozostałe funkcje `add_item_quantity` i `decrease_item_quantity` zwiększają lub zmniejszają ilość podanego przedmiotu. Poniższe funkcje są używane w pozostałych skryptach.

```
extends Node2D
```

```
@onready var item_name
@onready var item_description
@onready var item_quantity
```

```

@onready var item_category
@onready var item_variable

func _ready() -> void:
    var random = randi() % 2
    if random == 0:
        item_name = "Orange"
    elif random == 1:
        item_name = "RedDiamond"

    $TextureRect.texture = load("res://assets/Items/" + item_name + ".png")
    var stack_size = int(JsonData.item_data[item_name]["StackSize"])
    item_quantity = randi() % stack_size + 1
    item_description = str(JsonData.item_data[item_name]["Description"])
    item_category = str(JsonData.item_data[item_name]["ItemCategory"])
    item_variable = int(JsonData.item_data[item_name]["ItemVariable"])
    if stack_size == 1:
        $Label.visible = false
    else:
        $Label.text = str(item_quantity)

func set_item(nm, qt, dc, ct, iv):
    print("setting item: ", nm, ", quantity: ", qt, ", with a description: ", dc)
    item_name = nm
    item_quantity = qt
    item_description = dc
    item_category = ct
    item_variable = iv
    $TextureRect.texture = load("res://assets/Items/" + item_name + ".png")

    var stack_size = int(JsonData.item_data[item_name]["StackSize"])
    if stack_size == 1:
        $Label.visible = false
    else:
        $Label.visible = true
        $Label.text = str(item_quantity)

func add_item_quantity(toAdd):
    item_quantity += toAdd
    $Label.text = str(item_quantity)

func decrease_item_quantity(toDec):
    item_quantity -= toDec
    $Label.text = str(item_quantity)

```

Poniżej znajduje się skrypt Slot.gd w listingu 10.8 odpowiedzialny za zachowanie rubryk zawartych w scenie Inventory. Funkcja `_ready`, w tym przypadku łączy odpowiednie sygnały i zapisuje tekstury reprezentujące czy dana rubryka jest wypełniona, `refresh_style` ustawia odpowiedni obraz wraz z każdą zmianą. Funkcje `pickFromslot` i `pushIntoslot` są używane w skrypcie Inventory do usunięcia i przypisania przedmiotu do rubryki w scenie Inventory. Funkcja `inititalize_item` wykorzystuje ze skryptu Item_tres.gd funkcje `set_item` do stworzenia przedmiotu w rubryce, a pozostałe funkcje `hovering_start` i `hovering_end` są wykorzystywane do pokazania sceny zawierającej opis przedmiotu.

```
extends Panel

signal hovering_started
signal hovering_ended

var taken = preload("res://assets/Items/Item_slot_taken.png")
var empty = preload("res://assets/Items/Item_slot.png")

var taken_style: StyleBoxTexture = null
var empty_style: StyleBoxTexture = null

const Item = preload("res://scripts/Inventory/Item_script.tres.gd")
var ItemClass = preload("res://scenes/Inventory/Item.tscn")
var item = null

var description = load("res://scenes/Inventory/Description.tscn")
var description_instance = description.instantiate()

var slot_index

@export var slotType: SlotType

enum SlotType{
    ITEMS,
    SHIRT,
    PANTS,
    SHOES,
    SWORD
}

func _ready() -> void:
    taken_style = StyleBoxTexture.new()
    empty_style = StyleBoxTexture.new()
    taken_style.texture = taken
    empty_style.texture = empty
```

```

        hovering_started.connect(hovering_start)
        hovering_ended.connect(hovering_end)
        #if randi() % 2 ==0:
        #    item = ItemClass.instantiate()
        #    add_child(item)
        refresh_style()

func refresh_style():
    if item==null:
        set('theme_override_styles/panel', empty_style)
    else:
        set('theme_override_styles/panel', taken_style)

func pickFromSlot():
    remove_child(item)
    var inv = find_parent("Inventory")
    inv.add_child(item)
    item = null
    refresh_style()

func putIntoSlot(new_item):
    item = new_item
    item.position = Vector2(0, 0)
    var inv = find_parent("Inventory")
    inv.remove_child(item)
    add_child(item)
    refresh_style()

func initialize_item(item_name, item_quantity, item_description,
item_category, item_variable):
    if item == null:
        item = ItemClass.instantiate()
        add_child(item)
        item.set_item(item_name, item_quantity, item_description,
item_category, item_variable)
    else:
        item.set_item(item_name, item_quantity, item_description,
item_category, item_variable)
        refresh_style()

func hovering_start():
    if item != null:
        if slotType == SlotType.ITEMS:
            description_instance.position = $".position
            var description_name =
description_instance.get_node("TextureRect/VBoxContainer/Name")
            var description_desc =

```

```

description_instance.get_node("TextureRect/VBoxContainer/Description")
    description_name.text = PlayerInventory.inventory[slot_index][0]
    description_desc.text = PlayerInventory.inventory[slot_index][2]
    var inv = find_parent("Inventory")
    inv.add_child(description_instance)
else:
    description_instance.position = $".".position
    var description_name =
description_instance.get_node("TextureRect/VBoxContainer/Name")
    var description_desc =
description_instance.get_node("TextureRect/VBoxContainer/Description")
    description_name.text = PlayerInventory.equips[slot_index][0]
    description_desc.text = PlayerInventory.equips[slot_index][2]
    var inv = find_parent("Inventory")
    inv.add_child(description_instance)

func hovering_end():
    await get_tree().create_timer(2.0).timeout
    var inv = find_parent("Inventory")
    inv.remove_child(description_instance)

```

Listing 10.8 Skrypt Slot.gd

Kolejnym skrypcem opisanym poniżej jest PlayerInventory.gd w listingu 10.9, który w zmiennych inventory i equips przechowuje dane mające reprezentować przedmioty zawarte w rubrykach. Funkcja add_item dodaje na podstawie parametru ilość przedmiotu zawartego w kontenerze oraz zapisuje zmiany do tabeli. Funkcja add_item_to_empty_slot zapisuje w tabeli nową instancję przedmiotu w tabeli inventory lub equips, natomiast remove_item pozbywa się tej instancji. Pozostałe funkcje add_item_quantity i update_slot_visual, dodają większą ilość przedmiotu w rubryce i zmieniają rubrykę, jeżeli jest w niej zawarty przedmiot.

```
extends Node
```

```

const SlotClass = preload("res://scripts/Inventory/slot.gd")
const ItemClass = preload("res://scripts/Inventory/Item_script.tres.gd")
const NUM_INVENTORY_SLOTS = 24

var inventory = {
    0: ["Orange", 1, "d", "Resource", 20],
    1: ["RedDiamond", 23, "d", "Resource", 0],
    2: ["RedDiamond", 95, "d", "Resource", 0]
}

```

```

var equips = {
    0: ["The Coat", 1, "d", "Shirt", 2],
    1: ["The Pants", 1, "d", "Pants", 2],
    2: ["The Boots", 1, "d", "Shoes", 2],
    3: ["The Sword", 1, "Imagination tailored", "Sword", 5]
}

var inventory_ui = null

func add_item(item_name, item_quantity, item_description, item_category,
item_variable):
    #print("add_item: ", item_name, ", quantity: ", item_quantity)
    for item in inventory:
        if inventory[item][0] == item_name:
            var stack_size = int(JsonData.item_data[item_name]["StackSize"])
            var able_to_add = stack_size - inventory[item][1]
            if able_to_add >= item_quantity:
                inventory[item][1] += item_quantity
                update_slot_visual(item, inventory[item][0],
inventory[item][1], inventory[item][2], inventory[item][3],
inventory[item][4])
                return
            else:
                inventory[item][1] += able_to_add
                update_slot_visual(item, inventory[item][0],
inventory[item][1], inventory[item][2], inventory[item][3],
inventory[item][4])
                item_quantity = item_quantity - able_to_add
                #TODO Check if slot is full
                #inventory[item][1] += item_quantity
                #return

    # PlayerInventory.gd

    for i in range(NUM_INVENTORY_SLOTS):
        if inventory.has(i) == false:
            # MUSISZ dodać wszystkie parametry, inaczej UI się pogubi
            inventory[i] = [item_name, item_quantity, item_description,
item_category, item_variable]

            # Kluczowy krok: Powiadom UI o zmianie
            update_slot_visual(i, inventory[i][0], inventory[i][1],
inventory[i][2], inventory[i][3], inventory[i][4])
            return

func add_item_to_empty_slot(item: ItemClass, slot: SlotClass):

```



```

    match slot.SlotType:
        SlotClass.SlotType.SHIRT or SlotClass.SlotType.PANTS or
SlotClass.SlotType.SHOES or SlotClass.SlotType.SWORD:
            equips[slot.slot_index] = [item.item_name, item.item_quantity,
item.item_description, item.item_category, item.item_variable]
        _:
            inventory[slot.slot_index] = [item.item_name,
item.item_quantity, item.item_description, item.item_category,
item.item_variable]

```

```

func remove_item(slot: SlotClass):

```

```

    match slot.SlotType:
        SlotClass.SlotType.SHIRT or SlotClass.SlotType.PANTS or
SlotClass.SlotType.SHOES or SlotClass.SlotType.SWORD:
            equips.erase(slot.slot_index)
        _:
            inventory.erase(slot.slot_index)

```

```

func add_item_quantity(slot: SlotClass, quantity_to_add: int):
    match slot.SlotType:
        SlotClass.SlotType.SHIRT or SlotClass.SlotType.PANTS or
SlotClass.SlotType.SHOES or SlotClass.SlotType.SWORD:
            equips[slot.slot_index][1] += quantity_to_add
        _:
            inventory[slot.slot_index][1] += quantity_to_add

```

```

# PlayerInventory.gd

```

```

func update_slot_visual(slot_index, item_name, new_quantity, item_desc,
item_cat, item_var):
    # Sprawdzamy, czy scena Inventory jest w ogóle otwarta
    if inventory_ui != null:
        # Tutaj precyzyjnie wskazujemy na GridContainer wewnątrz Twojej
sceny
        var grid = inventory_ui.get_node("GridContainer")
        var slots = grid.get_children()
        if slot_index < slots.size():
            var slot = slots[slot_index]
            if slot.item != null:
                slot.item.set_item(item_name, new_quantity, item_desc,
item_cat, item_var)
            else:
                slot.initialize_item(item_name, new_quantity, item_desc,
item_cat, item_var)

```

Listing 10.9 Skrypt PlayerInventory.gd

Skrypt implementujący wszystkie powyższe funkcje jest inventory.gd zawarty w listingu 10.10. Kontroluje wszelkie akcje jakie gracz wykonuje w scenie, w której zawarty jest kontener z ekwipunkiem. Funkcja `_ready` współpracująca z funkcjami `initialize_inventory()` i `initialize_equips()` tworzy w rubrykach przedmioty według tabel w `PlayerInventory.gd` oraz łączy rubryki z funkcją `slot_gui_input` odpowiadającą za reagowanie na wciśnięcie lewego klawisza myszy. W `slot_gui_input` znajdują się instrukcje warunkowe, gdzie na podstawie tego wykonywana jest reszta funkcji w `inventory.gd`. Funkcja `able_to_put_into_slot` służy do sprawdzenia czy przedmiot można umieścić do rubryki w ekwipunku, czy w zakładaniu. `Left_click_empty` jest funkcją umieszczającą przedmiot w rubryce, `left_click_diff_item` wstawia przedmiot przenoszony przez myszkę i przepisuje do niej inny typ przedmiotu. Funkcja `left_click_same_item` dodaje większą ilość do przedmiotu w rubryce, jeżeli on i przedmiot przepisany do myszki jest taki sam, natomiast `left_click_not_holding` usuwa przedmiot z rubryki i przypisuje do myszy.

```
extends Control
```

```
const SlotClass = preload("res://scripts/Inventory/slot.gd")
@onready var invent_slots = $GridContainer
@onready var equip_slots = $EquipSlots
var holding_item = null
var description = preload("res://scenes/Inventory/Description.tscn")

func _ready() -> void:
    var slots = invent_slots.get_children()
    var e_slots = equip_slots.get_children()
    for s in range(slots.size()):
        slots[s].gui_input.connect(slot_gui_input.bind(slots[s]))
        slots[s].slot_index = s
        slots[s].slotType = SlotClass.SlotType.ITEMS
    for i in range(e_slots.size()):
        e_slots[i].gui_input.connect(slot_gui_input.bind(e_slots[i]))
        e_slots[i].slot_index = i
    e_slots[0].slotType = SlotClass.SlotType.SHIRT
    e_slots[1].slotType = SlotClass.SlotType.PANTS
    e_slots[2].slotType = SlotClass.SlotType.SHOES
    e_slots[3].slotType = SlotClass.SlotType.SWORD
    PlayerInventory.inventory_ui = self
    initialize_inventory()
    initialize_equips()
```

```

func initialize_inventory():
    var slots = invent_slots.get_children()
    for i in range(slots.size()):
        if PlayerInventory.inventory.has(i):
            slots[i].initialize_item(PlayerInventory.inventory[i][0],
PlayerInventory.inventory[i][1], PlayerInventory.inventory[i][2],
PlayerInventory.inventory[i][3], PlayerInventory.inventory[i][4])

func initialize_equips():
    var e_slots = equip_slots.get_children()
    for i in range(e_slots.size()):
        if PlayerInventory.equips.has(i):
            e_slots[i].initialize_item(PlayerInventory.equips[i][0],
PlayerInventory.equips[i][1], PlayerInventory.equips[i][2],
PlayerInventory.equips[i][3], PlayerInventory.equips[i][4])

func slot_gui_input(event: InputEvent, slot: SlotClass):
    if event is InputEventMouseButton:
        if event.button_index == MOUSE_BUTTON_LEFT && event.pressed:
            if holding_item != null:
                if !slot.item:
                    left_click_empty(slot)
                else:
                    # Different item, so swap
                    if holding_item.item_name != slot.item.item_name:
                        left_click_diff_item(event, slot)
                    # Same item, so try to merge
                    else:
                        left_click_same_item(slot)
            elif slot.item:
                left_click_not_holding(slot)
        elif event.button_index == MOUSE_BUTTON_RIGHT && event.pressed:
            if holding_item == null:
                if slot.item:
                    #slot.description_instance.position = Vector2(10,10)
                    #inv.add_child(slot.description_instance)
                    slot.hovering_started.emit()
                    slot.hovering_ended.emit()
                    #holding_item = slot.description_instance

func _input(event: InputEvent) -> void:
    if holding_item:
        holding_item.global_position = get_global_mouse_position()

func able_to_put_into_slot(slot: SlotClass):
    if holding_item == null:
        return true

```

```

    var holding_item_category =
JsonData.item_data[holding_item.item_name]["ItemCategory"]
    if slot.slotType == SlotClass.SlotType.SHIRT:
        return holding_item_category == "Shirt"
    elif slot.slotType == SlotClass.SlotType.PANTS:
        return holding_item_category == "Pants"
    elif slot.slotType == SlotClass.SlotType.SHOES:
        return holding_item_category == "Shoes"
    elif slot.slotType == SlotClass.SlotType.SWORD:
        return holding_item_category == "Sword"
    return true

func left_click_empty(slot: SlotClass):
    if able_to_put_into_slot(slot):
        PlayerInventory.add_item_to_empty_slot(holding_item, slot)
        slot.putIntoSlot(holding_item)
        holding_item = null

func left_click_diff_item(event: InputEvent, slot: SlotClass):
    if able_to_put_into_slot(slot):
        PlayerInventory.remove_item(slot)
        PlayerInventory.add_item_to_empty_slot(holding_item, slot)
        var temp_item = slot.item
        slot.pickFromSlot()
        temp_item.global_position = event.global_position
        slot.putIntoSlot(holding_item)
        holding_item = temp_item

func left_click_same_item(slot: SlotClass):
    if able_to_put_into_slot(slot):
        var stack_size =
int(JsonData.item_data[slot.item.item_name]["StackSize"])
        var able_to_add = stack_size - slot.item.item_quantity
        if able_to_add >= holding_item.item_quantity:
            PlayerInventory.add_item_quantity(slot,
holding_item.item_quantity)
            slot.item.add_item_quantity(holding_item.item_quantity)
            holding_item.queue_free()
            holding_item = null
        else:
            PlayerInventory.add_item_quantity(slot, able_to_add)
            slot.item.add_item_quantity(able_to_add)
            holding_item.decrease_item_quantity(able_to_add)

func left_click_not_holding(slot: SlotClass):
    PlayerInventory.remove_item(slot)
    holding_item = slot.item
    slot.pickFromSlot()

```

```
holding_item.global_position = get_global_mouse_position()
```

Listing 10.10 Skrypt inventory.gd

10.5. Menadżer stanów postaci gracza

Rozwiązanie znane jako maszyna stanów jest bardzo popularne, ze względu na estetykę kodu i łatwe pozbywanie się błędów oraz ich zapobieganie. Stany w tym przypadku rozumiane są jako różne sposoby wykonywania tego samego cyklicznego procesu obsługującego takie rzeczy jak ruch postaci czy wywoływanie jej animacji. Na potrzeby projektu autorzy postanowili posługiwać się gotowym rozwiązaniem dostępnym na stronie GDQuest [43], dosyć jednak zmodyfikowanym na potrzeby projektu.

```
## Virtual base class for all states.
## Extend this class and override its methods to implement a state.
class_name State extends Node

## Emitted when the state finishes and wants to transition to another state.
signal finished(next_state_path: String, data: Dictionary)

## Called by the state machine when receiving unhandled input events.
func handle_input(_event: InputEvent) -> void:
    pass

## Called by the state machine on the engine's main loop tick.
func update(_delta: float) -> void:
    pass

## Called by the state machine on the engine's physics update tick.
func physics_update(_delta: float) -> void:
    pass

## Called by the state machine upon changing the active state. The `data`
parameter
## is a dictionary with arbitrary data the state can use to initialize itself.
func enter(previous_state_path: String, data := {}) -> void:
    pass

## Called by the state machine before changing the active state. Use this
function
## to clean up the state.
func exit() -> void:
    pass
```

Listing 10.11. Kod pliku źródłowego State.gd

```
class_name StateMachine extends Node
```

```

@export var initial_state: State = null

@onready var state: State = (func get_initial_state() -> State:
    if DoorManager.did_player_go_through_doors:
        return get_child(1) #Getting out
    if initial_state != null :
        return initial_state
    else:
        return get_child(0) #idle
).call()

func _ready() -> void:
    for state_node: State in find_children("*", "State"):
        state_node.finished.connect(_transition_to_next_state)

    await owner.ready
    state.enter("")

func _unhandled_input(event: InputEvent) -> void:
    state.handle_input(event)

func _process(delta: float) -> void:
    state.update(delta)

func _physics_process(delta: float) -> void:
    state.physics_update(delta)

func _transition_to_next_state(target_state_path: String,
    data: Dictionary = {}) -> void:
    if not has_node(target_state_path):
        printerr(owner.name + ": Trying to transition to state "
            target_state_path + " but it does not exist.")
        return

    var previous_state_path := state.name
    state.exit()
    state = get_node(target_state_path)
    state.enter(previous_state_path, data)

```

Listing 10.12. Kod pliku źródłowego StateMachine.gd

Listing 10.11. Przedstawia klasę “State”, będącą bazą dla wszystkich klas stanów, wraz z definicjami jej metod, wspólnych dla każdej klasy potomnej. Te metody są z kolei

wywoływane w trakcie działania klasy “StateMachine” widocznej na listingu 10.12. Właśnie w niej przeprocesowane są właściwości fizyczne sceny postaci gracza, gdzie po stronie tej klasy wewnątrz jej funkcji “_process” i “process_physics” wywoływane są jedynie metody klasy “State”. Maszyna stanów obsługuje również sygnał zmiany stanu za pomocą funkcji “_transition_to_next_state” połączonej z tym sygnałem.

```
class_name PlayerState extends State

const IDLE = "Idle"
const RUNNING = "Running"
const JUMPING = "Jumping"
const FALLING = "Falling"
const WALKING = "Walking"
const GETTING_IN = "Getting_In"
const GETTING_OUT = "Getting_Out"

var player: Player

signal play_anim(animation_name)
signal change_orientation

func _ready() -> void:
    await owner.ready
    player = owner as Player
    assert(player != null, "The PlayerState state type must be
        used only in the player scene.
        It needs the owner to be a Player
        node.")
```

Listing 10.13. Kod pliku źródłowego StateMachine.gd

Klasa “PlayerState” z listingu o numerze 10.13 rozszerza klasę “State” z listingu 10.11. z uwagi na potrzebę zachowania odrębności unikalnych stanów gracza i cudzych stanów. Zdefiniowane są w niej stałe nazwy wszystkich dostępnych stanów później dostarczane do maszyny stanów, aby umożliwić jej zmianę stanu na inny wybrany jako argument sygnału.

```
extends PlayerState
```



```

func enter(previous_state_path: String, data := {}) -> void:
    player.velocity.x = 0.0
    play_anim.emit("idle")

func physics_update(_delta: float) -> void:
    player.velocity.y += player.gravity * _delta
    player.move_and_slide()

    if GlobalVariables.is_idle_forced:
        pass
    elif
        DoorManager.is_player_inside_vertical_doors
        && Input.is_action_just_pressed("Interact")
        || DoorManager.is_player_inside_horizontal_doors
    :
        finished.emit(GETTING_IN)
    elif not player.is_on_floor():
        finished.emit(FALLING)
    elif Input.is_action_just_pressed("Jumping")
        && player.is_jumping_allowed:
        finished.emit(JUMPING)
    elif Input.is_action_pressed("Move_Left")
        or Input.is_action_pressed("Move_Right"):
        if Input.is_action_pressed("Running"):
            finished.emit(RUNNING)
        else:
            finished.emit(WALKING)

```

Listing 10.14. Kod przykładowego stanu “Idle” w idle.gd.

Listing 10.14 przedstawia metody wykonywane w stanie “Idle”, w tym “enter” wywoływane za każdym razem przy wejściu do danego stanu. Natomiast “physics_update” wykonywane jest w maszynie stanów za w każdej klatce działania programu. Obsługują one szybkość postaci gracza, wpływ grawitacji na nią oraz emitowanie sygnału o zmianie animacji wykonywanej w scenie postaci. Na sam koniec klasy “physics_update” sprawdzane są warunki przejścia z danego stanu na inny, jeśli jakiś jest spełniony emitowany jest sygnał połączony z maszyną stanów, która z kolei zmienia obsługiwany stan na proszony w argumencie sygnału.

```

extends PlayerState

```

```

func enter(previous_state_path: String, data := {}) -> void:

```

```

play_anim.emit("running")

func physics_update(delta: float) -> void:
    var input_direction_x := Input.get_axis("Move_Left", "Move_Right")
    player.velocity.x = 1.3 * player.speed * input_direction_x
    player.velocity.y += player.gravity * delta

    if player.direction * input_direction_x < 0:
        player.direction *= -1
        change_orientation.emit(0)

    player.move_and_slide()

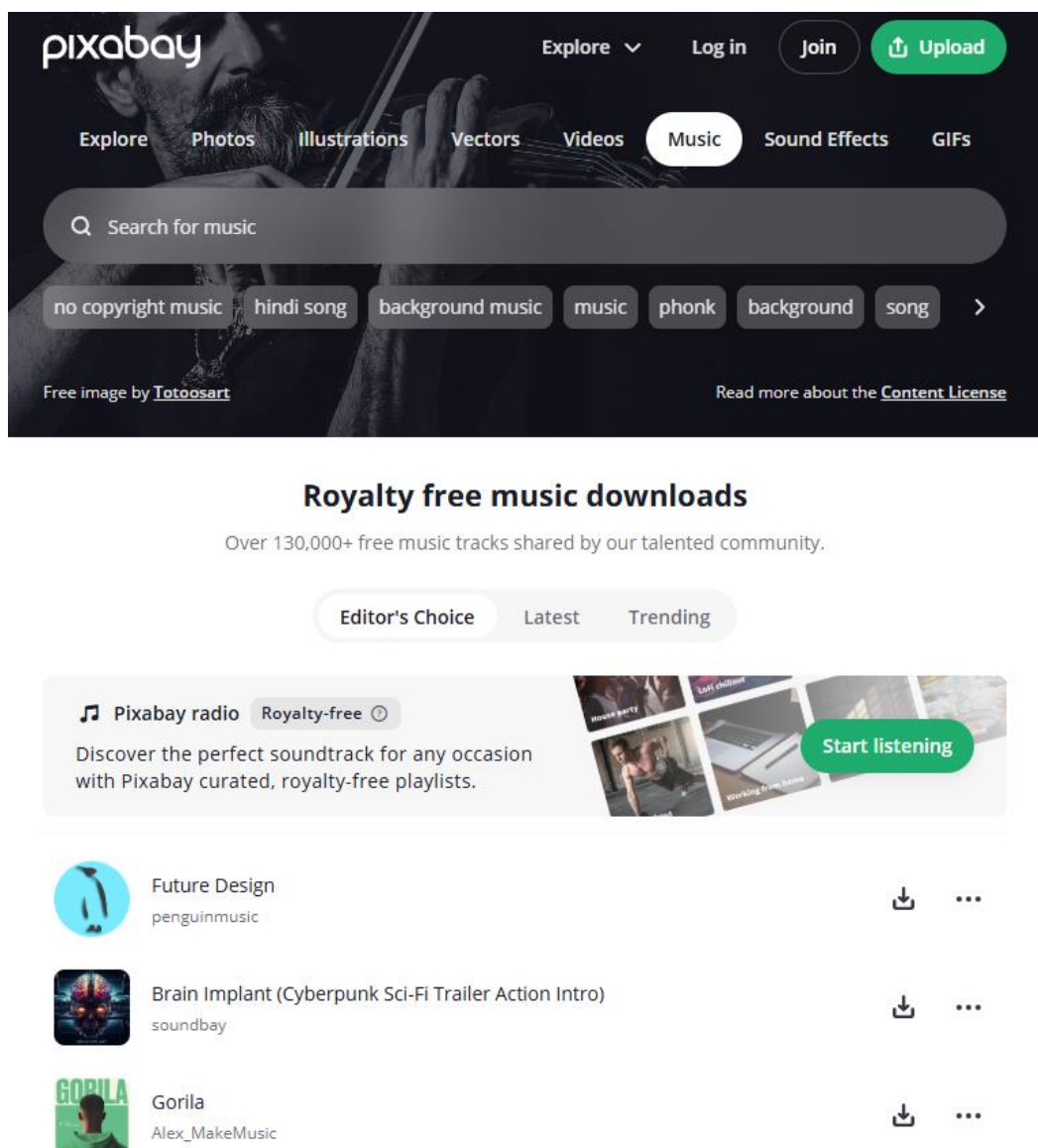
    if GlobalVariables.is_idle_forced:
        finished.emit(IDLE)
    elif DoorManager.is_player_inside_vertical_doors &&
Input.is_action_just_pressed("Interact") ||
DoorManager.is_player_inside_horizontal_doors:
        finished.emit(GETTING_IN)
    elif not player.is_on_floor():
        finished.emit(FALLING)
    elif Input.is_action_just_pressed("Jumping") &&
player.is_jumping_allowed:
        finished.emit(JUMPING)
    elif is_equal_approx(input_direction_x, 0.0):
        finished.emit(IDLE)
    elif !Input.is_action_pressed("Running"):
        finished.emit(WALKING)

```

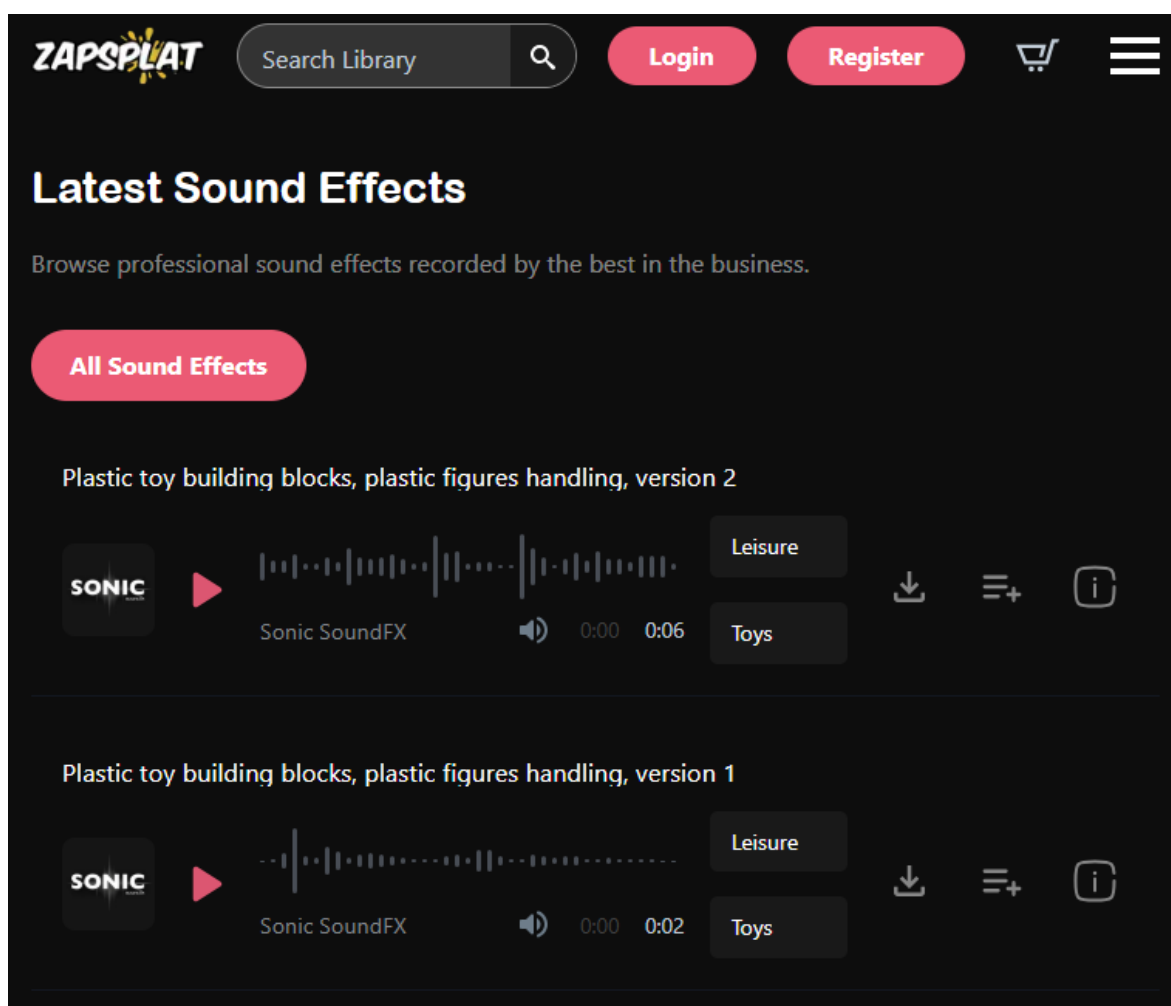
Listing 10.15. Kod przykładowego stanu “Running” w running.gd.

11. Elementy dźwiękowe

Gra wykorzystuje darmowe treści dostępne w internetowych bibliotekach, przykładowo Pixabay [44] (rys. 11.1), zawierające muzykę lub Zapslat [45] (rys. 11.2) koncentrujący efekty dźwiękowe. Są to bogate źródła treści audio z dużą gamą gatunków muzycznych i dźwięków w stylistyce rozmaitych gier komputerowych.



Rys 11.1. Screenshot ze strony Pixabay



Rys 11.2. Screenshot ze strony Zapsplat

Są one popularne wśród twórców ze względu na możliwość całkowicie darmowego użytkowania, jednak dużą elastyczność daje również przygotowywanie treści na własną rękę. Jest to dobre rozwiązanie ze względu na dostosowywanie brzmienia na własne potrzeby bez potrzeby szukania jednego dźwięku, który idealnie pasuje. Takie podejście oferuje również możliwość modyfikacji darmowych treści z Internetu na własne potrzeby za pomocą technik wykorzystywanych przy tworzeniu własnego audio. Podstawowym narzędziem do tworzenia własnego audio są różnego rodzaju syntezatory, przykładowo Vital [46] (rys.11.3), będący popularną darmową opcją. Oferuje on rozmaite rodzaje syntezy dźwięku, następnie modulacji, a nawet nakładania własnych efektów dźwiękowych w obrębie jednego programu.



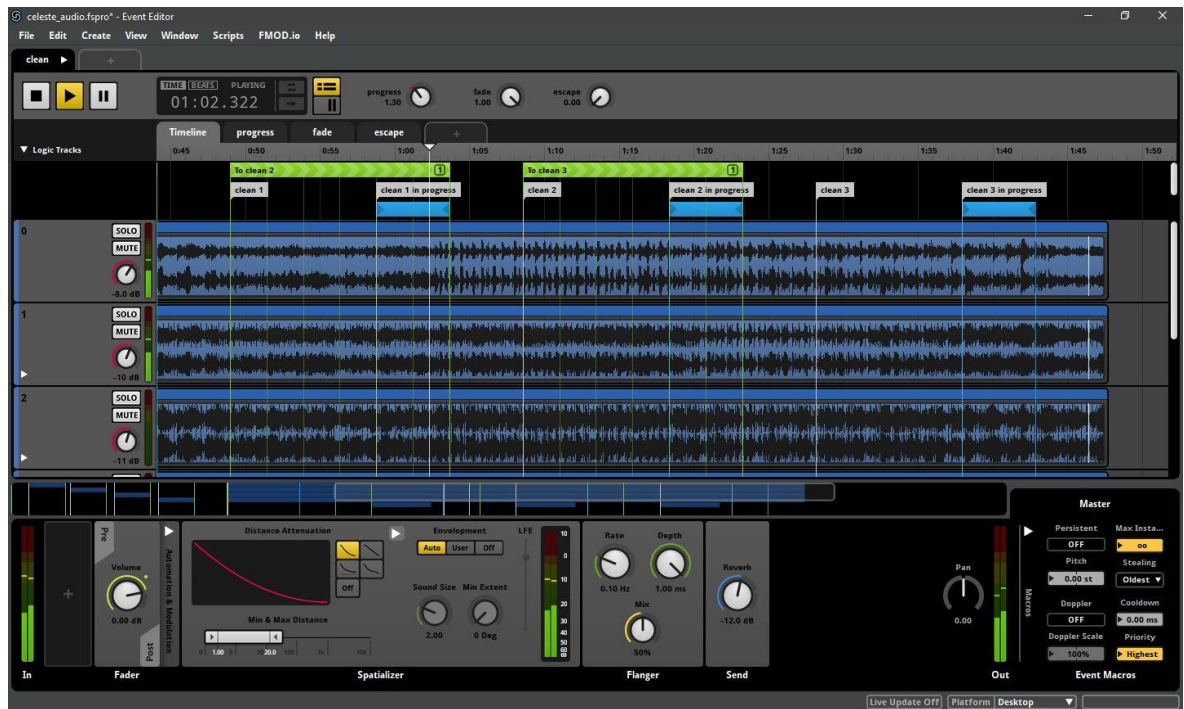
Rys 11.3. Screenshot z programu Vital.

Następnie po wydobyciu się dźwięku z tego typu programu audio jest on modyfikowany przez różne efekty takie jak pogłos, kompresja, echo czy przester pokazane na rysunku 11.4.



Rys 11.4. Przykładowe efekty dźwiękowe

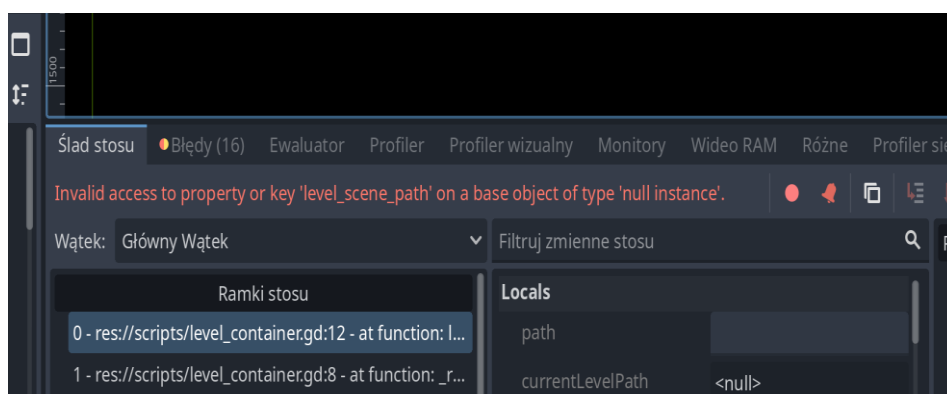
Większość twórców w dzisiejszych czasach używa tak zwane “middleware” [47] do implementowania oprawy dźwiękowej gier komputerowych. Są to programy zapewniające intuicyjne zarządzanie dźwiękiem w grze w zaawansowany sposób bez potrzeby pisania własnego skomplikowanego kodu. Przykładowym programem tego typu jest FMod [24] (rys. 11.5), będącym jednym z najpopularniejszych tego typu rozwiązań na rynku ze względu na przyjazność względem niezależnych twórców. Umożliwia on zarządzanie treściami audio poprzez przykładowo wyrównywanie poziomów ich głośności czy nakładanie rozmaitych efektów.



Rys 11.5. Screenshot interfejsu programu FMOD

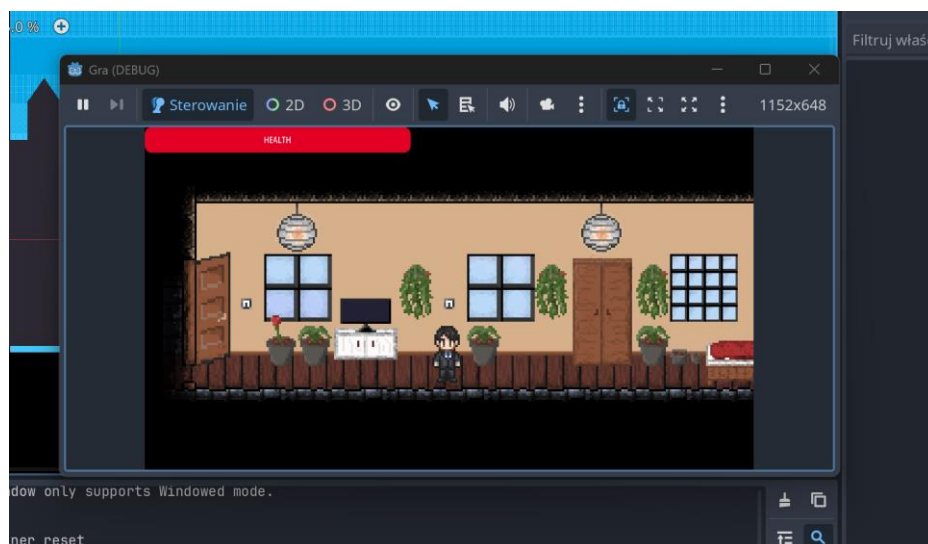
12. Testy i debugowanie

Podczas tworzenia gry były przeprowadzane testy mające sprawdzić poprawność działania programu. Pierwszą częścią było testowanie pojedynczych scen lub interfejsów, aby sprawdzić czy zawarte w nich skrypty wykonywały funkcje według oczekiwań twórców. Dzięki temu w przypadku znalezienia błędów w skrypcie (rys. 12.1) można je, natychmiastowo naprawić.

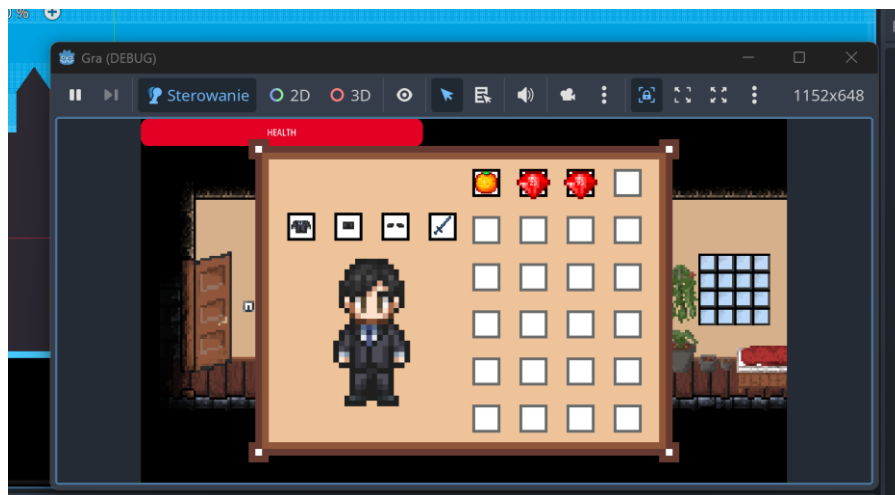


Rys. 12.1. Przykład błędu znalezionego w skrypcie.

Drugą częścią jest sprawdzenie jak owa scena współgra z innymi podczas grania (rys. 12.2 i 12.3). Najczęstszym napotkanym błędem w takich testach są źle włączające się interfejsy, czy też wartość null w zmiennej lub w powołanym obiekcie. Testy te są wykonywane, by sprawdzić czy interfejsy się poprawnie uruchamiają, czy postać, którą użytkownik steruje wykonuje poprawne ruchy, czy postaci wykonują funkcje do nich przepisane lub czy guziki poprawnie działają.



Rys. 12.2. Testowanie ruchu postaci podczas gry.



Rys. 12.3. Testowanie interfejsów podczas grania.

13. Wnioski

Cała implementacja gry w silniku Godot [6] przeszła pomyślnie i każdy krok został wykonany według oczekiwań twórców. Każdy element gry współpracuje z innymi, żeby odwzorować doświadczenie gry przygodowej z elementami rozgrywki platformowej.

Podczas tworzenia wystąpiły pewne przeszkody związane z brakiem doświadczenia z programem, ale na całe szczęście udało się je naprawić poprzez przejrzanie dokumentacji języka GDScript [7]. Największym problem była implementacja ekwipunku i menadżera scen, przez to, że każda zmienna w Godocie jest obiektowa wiele razy trzeba je sprawdzać, żeby nie natrafić na wartość null, w którejś z nich. Nie wspominając o instancjonowaniu scen w innych scenach i wstawiając ich jako węzły.

Są pewne elementy w grze, które mogłyby być lepiej zrobione jak efekty na postaciach podczas walki, grafika zaimplementowana do scen i postaci oraz bardziej rozwinięty interfejs. Jednakże w obecnym stanie gra wypełnia wszystkie kryteria związane z fabułą i mechaniką oraz zapewnia dużo zabawy.

Bibliografia

- [1] Kowert R., Quandt T. (2016). The Video Game Debate Unravelling the Physical, Social, and Psychological Effects of Video Games - <https://searchworks.stanford.edu/view/11418482>
- [2] Schneider E., Lang A., Shin M., Bradley S. - Death with a story: How story impacts emotional, motivational, and physiological responses to first-person shooter video games - <https://www.scopus.com/pages/publications/4043174880?origin=resultslist>
- [3] Mario - <https://pl.wikipedia.org/wiki/Mario>
- [4] Fear & Hunger - https://en.wikipedia.org/wiki/Fear_%26_Hunger
- [5] Earthbound - <https://pl.wikipedia.org/wiki/EarthBound>
- [6] Godot Engine – Dokumentacja silnika na oficjalnej stronie: <https://docs.godotengine.org/en/stable/>
- [7] GDscript – Oficjalna dokumentacja języka programowania: <https://docs.godotengine.org/en/stable/tutorials/scripting/gdscript/>
- [8] C++ - <https://pl.wikipedia.org/wiki/C%2B%2B>
- [9] C# - <https://learn.microsoft.com/pl-pl/dotnet/csharp/tour-of-csharp/>
- [10] Wang W., Walcott T. (2024). Programming for Game Design, A Hands-On Guide with Godot - <https://www.scopus.com/pages/publications/105001899708?origin=resultslist>
- [11] Kavanagh W., Miller A., Norman G., Andrei O. (2014) - Balancing Turn-Based Games With Chained Strategy Generation, <https://ieeexplore.ieee.org/document/8846763>
- [12] SteamDB - <https://steamdb.info/>
- [13] Celeste - <https://store.steampowered.com/app/504230/Celeste/>
- [14] Geometry Dash - <https://play.google.com/store/apps/details?id=com.robtopx.geometryjump&hl=pl>
- [15] Hollow Knight - https://store.steampowered.com/app/367520/Hollow_Knight/
- [16] Blasphemous - <https://store.steampowered.com/app/774361/Blasphemous/>
- [17] Stardew Valley - https://store.steampowered.com/app/413150/Stardew_Valley/
- [18] Terraria - <https://www.terraria.org/>
- [19] Project Zomboid - <https://projectzomboid.com/blog/>
- [20] Elden Ring - <https://en.bandainamcoent.eu/elden-ring/elden-ring>
- [21] Skyrim - https://store.steampowered.com/app/489830/The_Elder_Scrolls_V_Skyrim_Special_Edition/
- [22] Cyberpunk 2077 - <https://www.cyberpunk.net/pl/pl/>

- [23] Kingdom Come: Deliverance II - https://store.steampowered.com/app/1771300/Kingdom_Come_Deliverance_II/
- [24] Unity - <https://unity.com/>
- [25] Unreal - <https://www.unrealengine.com/en-US>
- [26] Virtual Cottage - <https://duandigames.com/our-games/virtual-cottage.html>
- [27] Undertale - <https://store.steampowered.com/app/391540/Undertale/>
- [28] RPG Maker MV - <https://www.rpgmakerweb.com/products/rpg-maker-mv>
- [29] Steam - <https://pl.wikipedia.org/wiki/Steam>
- [30] Fear and Hunger 2: Termina - https://store.steampowered.com/app/2171440/Fear_Hunger_2_Termina/
- [31] Sally Face - https://store.steampowered.com/app/541570/Sally_Face_Episode_One/
- [32] Nintendo Switch - <https://nintendo.pl/nintendo-switch/>
- [33] Xbox One - https://pl.wikipedia.org/wiki/Xbox_One
- [34] Playstation 4 - <https://www.playstation.com/pl-pl/ps4/>
- [35] Windows - https://www.geeksforgeeks.org/ethical-hacking/windows-os-fundamentals/?utm_source=chatgpt.com
- [36] Linux - <https://www.linux.com/what-is-linux/>
- [37] FMOD - <https://www.fmod.com/studio/>
- [38] Python - <https://docs.python.org/3/tutorial/index.html>
- [39] Visual Studio Code - <https://code.visualstudio.com/>
- [40] Aseprite - <https://libresprite.github.io/#/>
- [41] OpenGameArt - https://opengameart.org/art-search-advanced?keys=&field_art_type_tid%5B0%5D=9&sort_by=count&sort_order=DESC&page=2
- [42] Krita - <https://krita.org/en/>
- [43] GDQuest - <https://www.gdquest.com/tutorial/godot/design-patterns/finite-state-machine/>
- [44] Pixabay - <https://pixabay.com/pl/>
- [45] Zapsplat - <https://www.zapsplat.com/>
- [46] Vital - <https://vital.audio/>
- [47] Middleware - https://pl.wikipedia.org/wiki/Oprogramowanie_po%C5%9Brednicz%C4%85ce

